ДИСЦИПЛИНА	Технологии разработки приложений на базе фреймворков
ИНСТИТУТ	Институт перспективных технологий и индустриального программирования
КАФЕДРА	Индустриального программирования
ВИД УЧЕБНОГО МАТЕРИАЛА	Практическая работа
ПРЕПОДАВАТЕЛЬ	Ящун Татьяна Викторовна
СЕМЕСТР	5 семестр, 2026-2027

ПРАКТИЧЕСКАЯ РАБОТА № 9
МОДЕЛИ DJANGO, СВЯЗИ И ОПЕРАЦИИ С БАЗОЙ ДАННЫХ ЧЕРЕЗ ORM

ЦЕЛЬ РАБОТЫ

Изучить принципы описания моделей Django и связей между ними, освоить создание и применение миграций для управления структурой базы данных, а также выполнение операций создания, чтения, изменения и удаления записей средствами Django ORM.

Применить полученные знания в индивидуальном сквозном проекте: перевести слой доступа к данным с ручных SQL-запросов и библиотеки sqlite3 на Django ORM, сохранив работоспособность реализованных ранее страниц и пользовательских сценариев.

СОДЕРЖАНИЕ

1. ЗАДАЧИ РАБОТЫ 3
2. ПОРЯДОК ВЫПОЛНЕНИЯ И ОФОРМЛЕНИЯ РАБОТЫ 4
  2.1. Подготовка к работе 4
  2.2. Выполнение материалов Яндекс Практикума 4
  2.3. Работа над индивидуальным проектом 4
  2.4. Контрольная точка Git 5
  2.5. Требования к выполнению и оформлению 5
3. ТЕОРЕТИЧЕСКИЕ СВЕДЕНИЯ 7
  3.1. Django ORM и реляционная база данных 7
  3.2. Модель и таблица базы данных 8
  3.3. Связи между моделями 8
  3.4. Миграции 9
  3.5. CRUD-операции через Django ORM 10
  3.6. Заполнение и выгрузка данных 10
  3.7. Переход проекта от sqlite3 к Django ORM 11
4. ПРАКТИЧЕСКАЯ ЧАСТЬ. ВЫПОЛНЕНИЕ МАТЕРИАЛОВ ЯНДЕКС ПРАКТИКУМА 12
  4.1. Общий порядок работы 12
  4.2. Блок «Отношения между таблицами» 13

4.2.1. Урок «Отношения между таблицами» 13
4.2.2. Урок «Отношения „один к одному“» 13
4.2.3. Урок «Отношения „многие к одному“» 14
4.2.4. Урок «Объединение таблиц: JOIN» 14
4.2.5. Урок «Отношения между таблицами. Многие ко многим» 14
4.2.6. Урок «Изменение таблиц в БД. Ссылочная целостность» 15
4.3. Блок «Django ORM. Модели» 15
4.3.1. Урок «Описание таблиц с помощью классов» 16
4.3.2. Урок «Описание связей между моделями» 16
4.3.3. Урок «Наследование от абстрактных моделей» 17
4.3.4. Урок «Миграции» 17
4.3.5. Урок «Django shell. CRUD-операции в Django ORM» 17
4.3.6. Урок «Заполнение БД и выгрузка информации из неё» 18
4.4. Результаты изучения раздела 19
5. ИНДИВИДУАЛЬНЫЙ СКВОЗНОЙ ПРОЕКТ 19
5.1. Общие требования к проекту 19
5.2. Архитектура создаваемого проекта 20
5.3. Анализ существующей базы данных 23
5.4. Выбор стратегии перехода к Django ORM 25
5.5. Создание моделей Django 26
5.5.1. Модель помещения Room 26
5.5.2. Модель пользователя User 27
5.5.3. Модель бронирования Booking 28
5.6. Проверка регистрации приложений и моделей 31
5.7. Создание и применение миграций 31
5.8. Перенос данных из базы ПР7 32
5.8.1. Подготовка к переносу 33
5.8.2. Реализация переноса 35
5.8.3. Проверка перенесённых данных 38
5.9. Реализация операций с данными через ORM 39
5.9.1. Получение списка помещений 39
5.9.2. Получение одного помещения 40
5.10. Использование связей между моделями 40
5.11. Адаптация представлений помещений 41
5.12. Адаптация представлений бронирований 47
5.13. Фильтрация списка бронирований по параметрам запроса 54
5.14. Сортировка списка бронирований 58
5.15. Пагинация списка бронирований 60
5.16. Проверка и сохранение изменений 62
5.17. Актуализация файла README.md 63
6. РЕЗУЛЬТАТ ВЫПОЛНЕНИЯ ПРАКТИЧЕСКОЙ РАБОТЫ 65
6.1. Проверка моделей и связей 66
6.2. Проверка миграций 67
6.3. Проверка операций ORM 67
6.4. Проверка пользовательских сценариев 67
6.5. Проверка соответствия кода требованиям 68
7. ФОРМА КОНТРОЛЯ И КРИТЕРИИ ОЦЕНКИ 68
7.1. Чек-лист выполнения материалов Яндекс Практикума 69
7.2. Чек-лист выполнения индивидуального проекта 69
7.3. Таблица оценивания индивидуального проекта 70
7.4. Требования к отчету 74
8. КОНТРОЛЬНЫЕ ВОПРОСЫ 75
9. ИТОГ 76

1. ЗАДАЧИ РАБОТЫ

В ходе выполнения практической работы необходимо решить следующие задачи:

1. Актуализировать знания о моделях Django, полях моделей, связях между моделями и механизме объектно-реляционного отображения (ORM).
2. Разработать или доработать модели данных индивидуального сквозного проекта с учётом предметной области, состава сущностей и связей между ними.
3. Реализовать связи между моделями с использованием `ForeignKey` и соответствующих параметров, включая `on_delete` и `related_name`, если они необходимы.
4. Создать и применить миграции для формирования и изменения структуры базы данных.
5. Освоить основные операции ORM: получение записей, создание, изменение и удаление объектов, фильтрацию, исключение записей и получение отдельных объектов.
6. Реализовать выборку и обработку данных по полям связанных моделей, а также сортировку и ограничение состава возвращаемых данных.
7. Реализовать поиск по пользовательскому запросу и фильтрацию данных по параметрам, передаваемым через HTTP-запрос.
8. Освоить получение связанных объектов и оптимизацию запросов с использованием `select_related()` и других подходящих средств ORM.
9. Реализовать агрегирование и группировку данных с использованием `annotate()`, `aggregate()`, `Count()`, `Sum()`, `Avg()`, `Min()`, `Max()` и других применимых средств ORM.
10. Реализовать постраничный вывод результатов с использованием `Paginator` и навигации между страницами.
11. Интегрировать ORM-запросы в представления Django и обеспечить передачу полученных данных в шаблоны для отображения пользователю.
12. Обеспечить корректную обработку некорректных параметров запроса, отсутствующих объектов и пустых результатов выборки.
13. Проверить работоспособность моделей, связей, запросов, представлений и шаблонов в составе индивидуального сквозного проекта.
14. Зафиксировать изменения в системе контроля версий и актуализировать документацию проекта.

Результат работы. Индивидуальный сквозной проект с моделями Django, корректными связями между ними и работающими ORM-запросами, обеспечивающими

получение, поиск, фильтрацию, сортировку, анализ и постраничное отображение данных.

2. ПОРЯДОК ВЫПОЛНЕНИЯ И ОФОРМЛЕНИЯ РАБОТЫ

Практическая работа выполняется последовательно по двум взаимосвязанным направлениям: заданиям Яндекс Практикума и индивидуальному сквозному проекту. Сначала изучаются материалы раздела 05 «База данных и Django ORM», затем изученные подходы применяются при доработке индивидуального проекта.

2.1. Подготовка к работе

Работа продолжается в существующем проекте, созданном в предыдущих практических работах. Изменения необходимо выполнять в существующем проекте: создание нового приложения вместо продолжения проекта, разработанного в ПР1–ПР7, не допускается. Перед началом работы:

- убедитесь, что проект после выполнения ПР7 запускается и содержит результаты предыдущих практических работ;
- создайте отдельную ветку или рабочее состояние проекта для выполнения ПР9;
- проверьте наличие Python и виртуального окружения;
- зафиксируйте текущее состояние проекта в Git (подраздел 2.4).

2.2. Выполнение материалов Яндекс Практикума

Изучите материалы раздела 05 «База данных и Django ORM» курса «Бэкенд-разработка» в Яндекс Практикуме в объёме, предусмотренном для ПР9. Материалы посвящены отношениям между таблицами реляционной базы данных, моделям Django, связям между ними, миграциям и CRUD-операциям.

Порядок выполнения материалов приведён в разделе 4, полный чек-лист – в подразделе 7.1.

2.3. Работа над индивидуальным проектом

Основная задача – заменить выполнение SQL-запросов непосредственно в представлениях Django на работу с базой данных посредством Django ORM. Предметная область, пользовательские сценарии, структура шаблонов и реализованная функциональность при этом сохраняются.

Последовательность доработки проекта:

1. Изучите структуру базы данных, созданной в ПР7, и определите соответствующие ей модели Django.
2. Опишите модели и связи между ними.
3. Настройте миграции и проверьте структуру базы данных.
4. Подготовьте данные для работы через ORM.
5. Реализуйте основные операции с данными средствами Django ORM.

6. Адаптируйте представления, ранее использовавшие ручные SQL-запросы и sqlite3.
7. Проверьте работу страниц, шаблонов, поиска и других ранее реализованных функций.
8. Обновите README.md, выполните проверки проекта и зафиксируйте результат в Git.

Подробные указания по каждому этапу приведены в разделе 5.

2.4. Контрольная точка Git

Перед началом работы необходимо зафиксировать состояние проекта. Проверьте состояние репозитория:

```
git status
```

Если в проекте остались незакоммиченные изменения, сначала завершите работу из ПР7 и сохраните корректное состояние проекта отдельным коммитом:

```
git add .
git commit -m "PR9: исходное состояние проекта перед переходом на ORM"
```

Коммиты, относящиеся к ПР9, должны начинаться с метки PR9:. Например:

```
PR9: описание моделей Room, User и Booking
PR9: миграции и перенос данных
PR9: переход представлений на ORM
```

2.5. Требования к выполнению и оформлению

При выполнении работы необходимо:

- использовать существующий проект и его предметную область;
- соблюдать принятую структуру каталогов и модулей;
- давать моделям и полям осмысленные имена;
- проверять работу каждого этапа до перехода к следующему;
- не удалять ранее реализованные функции без обоснованной необходимости;
- оформлять изменения структуры базы данных средствами миграций Django;
- сохранять миграции в системе контроля версий;
- не включать в репозиторий локальную базу данных и другие временные файлы, если они исключены правилами проекта;
- поддерживать актуальность README.md.

Команды, пути к файлам и примеры кода в индивидуальном проекте необходимо адаптировать к фактической структуре приложения.

К операциям с данными через ORM предъявляются минимальные требования по количеству и составу запросов (таблица 1):

Направление	Минимальное количество	Требования
Базовое получение данных	2	Получение наборов объектов и получение отдельного объекта по идентификатору или другому уникальному признаку
Фильтрация	4	Фильтрация по одному и нескольким условиям, фильтрация по полю связанной модели, исключение записей с помощью exclude()
Поиск	2	Поиск по строковому полю и поиск по нескольким полям либо по другому содержательному критерию предметной области
Сортировка	2	Сортировка по одному полю и по нескольким полям либо по полю связанной модели
Работа со связями	2	Получение данных с обращением к связанным моделям; применение select_related() или другого подходящего средства оптимизации, если это оправдано структурой моделей
Агрегирование и аннотирование	2	Получение сводного показателя и расчёт показателя для объектов или групп объектов с помощью aggregate() и/или annotate()
Постраничный вывод	1	Реализация выборки для страницы с использованием Paginator и корректной навигации по страницам
Итого	**Не менее 15 различных ORM-выражений**	**Запросы должны быть реализованы в проекте, проверены и использованы по назначению**

Указанные значения задают минимальные требования к представленным в проекте видам запросов. Если один и тот же ORM-запрос демонстрирует несколько приёмов, его можно учитывать в соответствующих тематических категориях, однако при подсчёте общего количества он считается одним запросом. Для выполнения общего требования необходимо реализовать не менее 15 различных ORM-выражений, а не просто 15 вызовов методов в одном составном выражении.

При реализации запросов необходимо соблюдать следующие требования:

1. Использовать имена моделей и полей, определённых в проекте.
2. Применять подходящие методы ORM: all(), filter(), exclude(), get(), order_by(), `values()`, `annotate()`, `aggregate()` и другие необходимые методы.
3. При фильтрации по связанным объектам использовать корректный синтаксис обращения к полям связанных моделей.
4. При поиске по строковым полям использовать подходящие операции, например `icontains`, `istartswith` или `iexact`, в зависимости от требований предметной области.
5. Если параметры фильтрации, поиска и сортировки передаются пользователем, предусмотреть их проверку. Для выбора поля сортировки использовать заранее определённое соответствие допустимых параметров полям модели, а не передавать произвольные пользовательские значения в `order_by()`.
6. При вычислении статистики использовать подходящие агрегатные функции и выражения ORM. Если проект предусматривает исключение отменённых, удалённых или иных неактуальных записей из расчётов, явно учитывать это условие.

7. При наличии объектов без связанных записей проверить, что они не исключаются из результата там, где должны отображаться; при необходимости использовать подходящий тип соединения или условие агрегирования.
8. При выборке связанных объектов оценить целесообразность применения `select_related()` или `prefetch_related()` для сокращения количества обращений к базе данных.
9. Не создавать дублирующие запросы с незначительно различающимися условиями только ради достижения требуемого количества.
10. Проверить корректность результатов на обычных, пустых и граничных наборах данных.

Запросы должны соответствовать реальным сценариям работы приложения: формированию списков объектов, поиску, применению фильтров, просмотру подробной информации, построению статистики и другим функциям предметной области.

3. ТЕОРЕТИЧЕСКИЕ СВЕДЕНИЯ

3.1. Django ORM и реляционная база данных

`Django ORM` (Object-Relational Mapping) – механизм, позволяющий работать с данными реляционной базы данных через объекты Python и классы моделей Django.

Без ORM приложение может выполнять SQL-запросы непосредственно через библиотеку работы с базой данных. Такой подход был использован в ПР7: SQL-запросы сохранялись в отдельных файлах (`sql/queries.sql`) или размещались в коде, а Django-представления выполняли их через модуль `sqlite3`.

Django ORM позволяет описывать структуру данных классами Python и выполнять типовые операции с помощью методов и атрибутов моделей. Например, вместо ручного SQL-запроса к таблице помещений приложение может использовать модель `Room` и выражение:

```
Room.objects.all()
```

ORM преобразует обращение к модели в SQL-запрос к базе данных и возвращает объекты, с которыми можно работать средствами Python.

Основные элементы Django ORM:

- **модель** – класс, описывающий сущность предметной области и её данные;
- **поле модели** – атрибут, соответствующий столбцу таблицы;
- **объект модели** – отдельная запись базы данных, представленная экземпляром класса;
- **менеджер objects** – интерфейс для выполнения запросов к модели;
- `QuerySet` – набор записей, полученный или подготовленный ORM-запросом;

- **миграция** – описание изменения структуры базы данных, сформированное на основе изменений моделей.

3.2. Модель и таблица базы данных

Модель Django обычно наследуется от `django.db.models.Model`. Её поля определяются специальными классами полей Django, например `CharField`, `IntegerField`, `BooleanField` или `DateField`.

Упрощённый пример модели помещения:

```
from django.db import models

class Room(models.Model):
    name = models.CharField(max_length=100)
    capacity = models.PositiveIntegerField()
```

В этом примере:

- `Room` – класс модели;
- `name` – поле для хранения названия помещения;
- `capacity` – поле для хранения вместимости;
- `max_length=100` задаёт максимальную длину строки для поля `name`.

По умолчанию Django добавляет первичный ключ, если он явно не определён в модели. Конкретный тип такого ключа зависит от настроек проекта.

Основные типы полей, используемые в моделях, приведены в таблице 2.

Тип поля	Назначение	Пример использования
`CharField`	строки ограниченной длины	название помещения
`TextField`	текстовые данные произвольной длины	описание помещения
`IntegerField`	целые числа	порядковый номер этажа
`PositiveIntegerField`	неотрицательные целые числа	вместимость помещения
`BooleanField`	логические значения	признак отмены бронирования
`DateField`	даты	дата бронирования
`DateTimeField`	дата и время	время создания записи

Тип поля необходимо выбирать исходя из назначения данных. Например, вместимость помещения может храниться в числовом поле, название – в строковом, а дата бронирования – в поле даты, если это соответствует требованиям проекта.

Модель не следует рассматривать только как описание таблицы. Она также предоставляет интерфейс для получения, создания, изменения и удаления записей.

3.3. Связи между моделями

Связи между моделями отражают отношения между сущностями предметной области. Основные виды связей:

- **один к одному** – одной записи соответствует не более одной записи другой модели; используется `OneToOneField`;

- **многие к одному** – несколько записей одной модели могут ссылаться на одну запись другой модели; используется `ForeignKey`;
- **многие ко многим** – записи одной модели могут быть связаны со множеством записей другой модели; используется `ManyToManyField`.

Для системы бронирования связь «многие к одному» подходит для отношения между бронированием и помещением: одно помещение может иметь несколько бронирований, но каждое бронирование относится к одному помещению. Аналогично бронирование связано с одним пользователем, а пользователь может создавать несколько бронирований.

Отношение «многие ко многим» в реляционной базе данных реализуется через промежуточную таблицу, содержащую внешние ключи к обеим связанным таблицам. Django может создать такую таблицу автоматически или использовать отдельную модель связи (параметр `through`), если сама связь обладает собственными характеристиками, например ролью участника или датой назначения.

При определении связей необходимо учитывать обязательность отношения, возможность отсутствия связанного объекта и поведение системы при удалении связанных записей (параметр `on_delete`).

3.4. Миграции

Миграции позволяют изменять структуру базы данных в соответствии с изменениями моделей Django. Создание моделей в Python ещё не создаёт соответствующие таблицы в базе данных: для этого необходимо сформировать и применить миграции.

Для создания файлов миграций используется команда `python manage.py makemigrations`. Она формирует описание изменений на основе текущих моделей и уже существующих миграций, но сама по себе не применяет эти изменения к базе данных.

Для применения миграций используется команда `python manage.py migrate`. После её выполнения изменения, описанные в миграциях, применяются к базе данных. Цикл изменения структуры базы данных показан на рис. 1.

Для изучения SQL, который соответствует конкретной миграции, можно применить команду `python manage.py sqlmigrate <app_label> <migration_name>`.

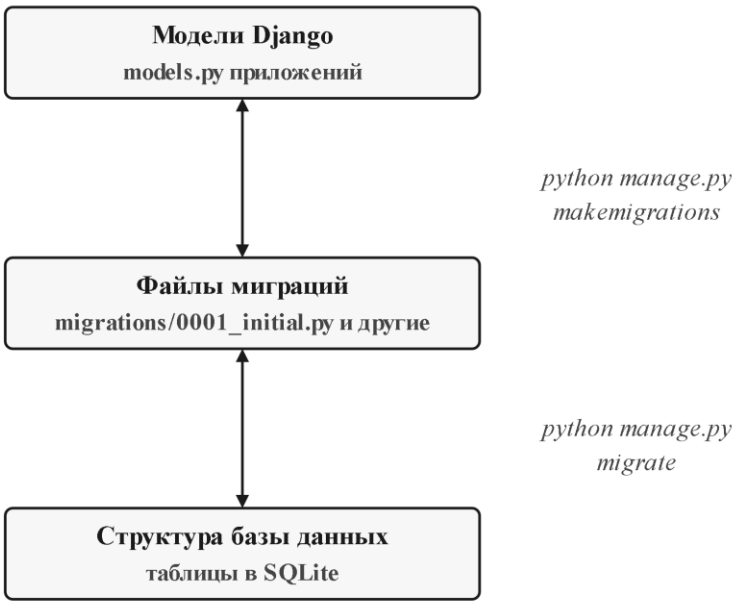


Рисунок 1 – Цикл изменения структуры базы данных

Файлы миграций являются частью проекта и должны сохраняться в Git. Это позволяет воспроизводить изменения структуры базы данных в других окружениях.

3.5. CRUD-операции через Django ORM

CRUD – четыре основные операции работы с данными. В Django ORM им соответствуют методы менеджера `objects` и экземпляров моделей (таблица 3).

Операция	Назначение	Пример Django ORM
Create	Создание записи	`Room.objects.create(...)`
Read	Получение записей	`Room.objects.all()`
Update	Изменение записи	изменение атрибутов объекта и вызов `save()`
Delete	Удаление записи	`room.delete()`

Для получения данных также применяются методы `filter()` и `get()`. Метод `filter()` возвращает `QuerySet`, содержащий записи, удовлетворяющие условиям. Метод `get()` предназначен для получения одного объекта и вызывает исключение, если подходящая запись отсутствует или найдено несколько записей.

При изменении отдельного объекта необходимо сохранить изменения вызовом `save()`. Для массового изменения записей может применяться метод `update()`, вызванный для `QuerySet`.

3.6. Заполнение и выгрузка данных

Django предоставляет средства для подготовки и переноса данных между окружениями.

Команда `python manage.py dumpdata` позволяет выгрузить данные моделей в формате `fixture`, а команда `python manage.py loaddata` – загрузить подготовленные данные в базу данных.

Эти средства полезны при подготовке тестовых данных, переносе данных между окружениями и восстановлении содержимого базы.

Перед загрузкой необходимо убедиться, что структура моделей и миграции в целевом окружении совместимы с загружаемыми данными. Загрузка fixture не заменяет миграции и не предназначена для изменения структуры таблиц: сначала миграции создают таблицы, затем `loaddata` заполняет их данными.

3.7. Переход проекта от sqlite3 к Django ORM

В ПР7 слой данных индивидуального проекта был реализован средствами SQL и стандартной библиотеки `sqlite3`: SQL-запросы хранились в файле `sql/queries.sql`, а представления выполняли их через `sqlite3.connect()` и курсор.

В ПР9 слой данных переводится на Django ORM: таблицы описываются моделями, структура базы управляется миграциями, а представления получают данные через менеджеры моделей. Цепочка работы с данными принимает вид:

```
Представление → Django ORM → SQLite → объекты моделей
```

Схема работы данных после перехода на ORM показана на рис. 2. Обращения к базе данных выполняются только средствами ORM: методы моделей формируют SQL-запросы автоматически, а результаты возвращаются в виде объектов.

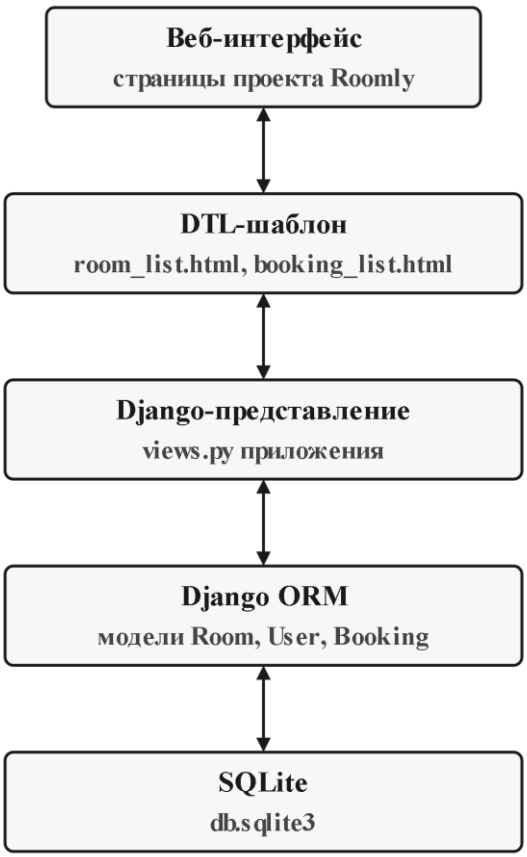


Рисунок 2 – Работа с данными через Django ORM

Изменение способа доступа к данным показано в таблице 4.

Аспект	Реализация в ПР7	Реализация в ПР9
Описание структуры	`sql/schema.sql`	модели в `models.py`
Изменение структуры	правка схемы вручную	миграции Django
Подключение к базе	`sqlite3.connect()`	менеджер `objects` модели
Получение данных	SQL-запросы из `sql/queries.sql`	`Room.objects.filter(...)` и другие методы
Связанные данные	`JOIN` в SQL-запросе	`select_related()`, обратная связь по внешним ключам
Результат запроса	словари `sqlite3.Row`	объекты моделей

Веб-часть `Roomly`, созданная в ПР5–ПР6, и пользовательские сценарии ПР7 сохраняются: изменяется способ получения данных в представлениях. Файлы SQL-слоя (`sql/schema.sql`, `sql/queries.sql`, `db/database.py`) остаются в проекте как предыдущая реализация предметной области, однако представления после перехода на ORM больше их не используют.

4. ПРАКТИЧЕСКАЯ ЧАСТЬ. ВЫПОЛНЕНИЕ МАТЕРИАЛОВ ЯНДЕКС ПРАКТИКУМА

Изучите материалы раздела 05 «База данных и Django ORM» курса «Бэкенд-разработка» в Яндекс Практикуме. Раздел посвящён отношениям между таблицами реляционной базы данных и работе с ними средствами Django ORM. Учебные задания выполняются до начала доработки индивидуального проекта и служат подготовкой к нему.

4.1. Общий порядок работы

Раздел состоит из двух тематических блоков:

- «Отношения между таблицами» – изучение реляционных связей, первичных и внешних ключей, объединения таблиц и ссылочной целостности;
- «Django ORM. Модели» – описание моделей Django, установление связей между ними, миграции, выполнение CRUD-операций и работа с данными.

При изучении каждого урока необходимо:

- ознакомиться с теоретическим материалом;
- выполнить предусмотренные учебные задания;
- проверить полученный результат;
- разобраться в назначении используемых конструкций и их влиянии на данные;
- определить, как изученный подход может быть применён в индивидуальном проекте.

Примечание. Учебные примеры курса могут использовать другую предметную область. Их назначение – продемонстрировать принцип работы. При выполнении индивидуального проекта необходимо применять изученные подходы к собственным сущностям и пользовательским сценариям.

4.2. Блок «Отношения между таблицами»

Блок посвящён реляционным связям: назначению ключей, типам отношений, объединению таблиц и ссылочной целостности.

4.2.1. Урок «Отношения между таблицами»

Изучите назначение первичных и внешних ключей, а также основные типы отношений между таблицами реляционной базы данных.

Необходимо разобраться:

- почему данные предметной области разделяют на несколько связанных таблиц;
- для чего используется первичный ключ;
- чем первичный ключ отличается от внешнего;
- как устанавливается связь между таблицами;
- почему внешний ключ должен ссылаться на существующую запись;
- как обязательность связи определяется ограничениями поля.

В учебных примерах обратите внимание на разделение данных об объектах предметной области: например, сведения о фильме, его оригинальном названии и типе могут храниться в разных таблицах. Такой подход позволяет не дублировать одни и те же данные во множестве записей.

Результат изучения: понимание назначения первичных и внешних ключей и умение определять, какие сущности предметной области должны храниться в отдельных таблицах.

4.2.2. Урок «Отношения „один к одному“»

Изучите отношение, при котором одной записи одной таблицы соответствует не более одной записи другой таблицы.

Разберите назначение ограничений:

- `PRIMARY KEY` – уникальная идентификация записи;
- `FOREIGN KEY` – ссылка на запись другой таблицы;
- `UNIQUE` – запрет повторения значения в поле;
- `NOT NULL` – запрет отсутствующего значения.

Обратите внимание, что внешний ключ сам по себе не всегда обеспечивает отношение «один к одному»: для этого необходимо исключить повторение соответствующего значения. Обязательность связи также определяется отдельно.

В Django такое отношение описывается полем `OneToOneField`.

Результат изучения: умение объяснить, чем отношение «один к одному» отличается от других типов связей и какие ограничения обеспечивают его корректность.

4.2.3. Урок «Отношения „многие к одному“»

Изучите отношение, при котором несколько записей одной таблицы могут ссылаться на одну запись другой таблицы. Например, несколько бронирований могут относиться к одному помещению: в таблице бронирований для этого хранится внешний ключ на таблицу помещений.

При изучении урока необходимо понять:

- в какой таблице размещается внешний ключ;
- почему его значение может повторяться;
- как отличить обязательную связь от необязательной;
- почему запись, на которую ссылаются другие записи, должна существовать.

В Django такое отношение обычно описывается полем `ForeignKey` на стороне модели, содержащей внешний ключ.

Результат изучения: умение определить сторону связи, в которой должен находиться внешний ключ, и объяснить, почему несколько бронирований могут ссылаться на одно помещение.

4.2.4. Урок «Объединение таблиц: JOIN»

Изучите способы получения данных из нескольких связанных таблиц с помощью SQL. Рассмотрите:

- назначение условия `ON`;
- использование `INNER JOIN`;
- отличие `INNER JOIN` от `LEFT JOIN`;
- назначение `RIGHT JOIN`, `FULL OUTER JOIN` и `CROSS JOIN`;
- использование псевдонимов столбцов и уточнение имён полей.

Особое внимание уделите различию между `INNER JOIN` и `LEFT JOIN`. При внутреннем соединении в результат попадают записи, для которых найдено соответствие в соединяемой таблице. При левом соединении сохраняются все записи левой таблицы, даже если соответствующих записей справа нет.

В ПР7 соединение таблиц использовалось для получения сведений о бронировании, помещении и пользователе. В ПР9 необходимо научиться получать связанные сведения через Django ORM.

Результат изучения: понимание принципа объединения связанных таблиц и умение объяснить, какие записи попадут в результат при использовании разных типов соединения.

4.2.5. Урок «Отношения между таблицами. Многие ко многим»

Изучите отношение, при котором одна запись может быть связана с несколькими записями другой таблицы и наоборот. В реляционной базе данных такая связь

обычно реализуется через промежуточную таблицу, содержащую внешние ключи, ссылающиеся на обе связанные таблицы.

Необходимо разобраться:

- почему связь «многие ко многим» нельзя корректно представить одним обычным внешним ключом;
- для чего нужна промежуточная таблица;
- как исключить повторение одной и той же пары связанных записей;
- в каких случаях промежуточная таблица должна содержать дополнительные поля.

В Django для такой связи используется `ManyToManyField`. Если сама связь обладает собственными характеристиками, например датой назначения участника на мероприятие или ролью участника, можно использовать отдельную модель и параметр `through`.

Результат изучения: умение определять необходимость связи «многие ко многим» и объяснять различие между автоматической промежуточной таблицей и отдельной моделью связи.

4.2.6. Урок «Изменение таблиц в БД. Ссылочная целостность»

Изучите, как ограничения базы данных обеспечивают согласованность связанных записей и какие проблемы могут возникать при изменении структуры таблиц. Рассмотрите:

- добавление и удаление таблиц;
- изменение структуры таблиц;
- попытку создать запись с несуществующим внешним ключом;
- удаление записи, на которую ссылаются другие записи;
- изменение значений ключевых полей;
- последствия нарушения ссылочной целостности.

В Django изменения структуры базы данных выполняются преимущественно через модели и миграции. В рамках ПР9 необходимо применять этот подход, а не изменять таблицы вручную командами SQL.

Результат изучения: понимание необходимости ссылочной целостности и умение объяснить, почему изменение связанных данных должно учитывать ограничения базы.

4.3. Блок «Django ORM. Модели»

Блок посвящён описанию таблиц классами моделей Django: типам полей, связям между моделями, наследованию, миграциям, CRUD-операциям и работе с данными.

4.3.1. Урок «Описание таблиц с помощью классов»

Изучите принцип преобразования моделей Django в таблицы реляционной базы данных.

Необходимо разобраться:

- почему модель наследуется от `models.Model`;
- как поля модели соответствуют столбцам таблицы;
- как выбирать тип поля с учётом хранимых данных;
- как задаются ограничения и значения по умолчанию;
- как Django определяет первичный ключ;
- как зарегистрировать приложение в настройках проекта.

Основные типы полей рассмотрены в подразделе 3.2: `CharField`, `TextField`, `IntegerField`, `PositiveIntegerField`, `BooleanField`, `DateField`, `DateTimeField`. Тип поля выбирается исходя из назначения данных: вместимость помещения хранится в числовом поле, название – в строковом, дата бронирования – в поле даты.

Результат изучения: умение описать сущность предметной области классом Django и подобрать поля, соответствующие её данным.

4.3.2. Урок «Описание связей между моделями»

Изучите, как отношения между таблицами представляются средствами Django ORM. Необходимо освоить назначение полей `OneToOneField`, `ForeignKey`, `ManyToManyField`.

Дополнительно изучите:

- параметр `on_delete`;
- параметр `related_name`;
- ссылку на модель по имени в виде строки;
- использование промежуточной модели для отношения «многие ко многим».

При выборе `on_delete` необходимо учитывать правила предметной области. Например, если удаление помещения не должно приводить к потере истории бронирований, поведение модели следует спроектировать соответствующим образом. Выбор `CASCADE`, `PROTECT`, `RESTRICT`, `SET_NULL` или другого варианта должен быть обоснован требованиями проекта.

Параметр `related_name` определяет имя обратного доступа к связанным объектам. Он позволяет получать связанные записи со стороны модели, на которую ссылаются.

Результат изучения: умение описать связи между моделями и объяснить, как через ORM получать связанные объекты.

4.3.3. Урок «Наследование от абстрактных моделей»

Изучите использование абстрактных моделей для повторного применения общих полей. Абстрактная модель может содержать атрибуты, необходимые нескольким моделям, например время создания и изменения записи. В отличие от обычной модели, абстрактная родительская модель сама по себе не создаёт отдельную таблицу.

Для объявления абстрактной модели используется внутренний класс `Meta` с атрибутом `abstract = True`.

Также изучите назначение полей с параметрами:

- `auto_now_add=True` – автоматическое заполнение времени создания записи;
- `auto_now=True` – автоматическое обновление времени при сохранении объекта.

Использование абстрактной модели в индивидуальном проекте является дополнительной задачей. Её следует применять, если несколько моделей действительно имеют общие поля и такое наследование делает структуру проекта понятнее.

Результат изучения: умение определить, когда абстрактная модель помогает избежать дублирования кода.

4.3.4. Урок «Миграции»

Изучите полный цикл изменения структуры базы данных через Django ORM. Назначение команд `python manage.py makemigrations` и `python manage.py migrate` рассмотрено в подразделе 3.4: первая формирует файлы миграций на основе изменений моделей, вторая применяет миграции к базе данных.

Дополнительно рассмотрите команду `python manage.py sqlmigrate <app_label> <migration_name>`: она позволяет посмотреть SQL, который соответствует конкретной миграции.

Обратите внимание на необходимость сохранять файлы миграций в Git. Файл локальной базы данных, например `db.sqlite3`, обычно не включают в репозиторий, если это предусмотрено правилами проекта.

Результат изучения: умение сформировать миграцию, применить её, проверить результат и объяснить разницу между созданием миграции и её применением.

4.3.5. Урок «Django shell. CRUD-операции в Django ORM»

Изучите выполнение операций с моделями вне веб-интерфейса приложения. Для запуска интерактивной оболочки используется команда `python manage.py shell`. В оболочке Django доступны модели проекта и их менеджеры: это позволяет проверять запросы, создавать тестовые записи и исследовать поведение моделей без запуска веб-сервера.

Освойте основные операции.

Создание записи:

```
Room.objects.create(
    name="Аудитория 301",
    capacity=30,
)
```

Получение записей:

```
Room.objects.all()
Room.objects.filter(capacity__gte=30)
```

Получение одного объекта:

```
Room.objects.get(pk=1)
```

Изменение объекта:

```
room = Room.objects.get(pk=1)
room.capacity = 35
room.save()
```

Удаление объекта:

```
room = Room.objects.get(pk=1)
room.delete()
```

Примечание. Примеры приведены для условной модели Room. Фактические названия моделей и полей необходимо согласовать с проектом.

При выполнении заданий обратите внимание на различие между `all()`, `filter()` и `get()`. Также необходимо понимать, что изменение атрибутов объекта не сохраняет данные в базе автоматически: для сохранения обычно требуется вызвать `save()`.

Результат изучения: умение выполнять основные CRUD-операции и проверять их результат через Django ORM.

4.3.6. Урок «Заполнение БД и выгрузка информации из неё»

Изучите использование fixtures для подготовки и переноса данных. Команда `python manage.py dumpdata` выгружает данные моделей, а команда `python manage.py loaddata` загружает данные из подготовленного fixture-файла.

Обратите внимание:

- структура моделей должна соответствовать загружаемым данным;
- миграции должны быть применены до загрузки;
- загрузка данных не заменяет создание таблиц;
- при переносе между окружениями необходимо учитывать зависимости между связанными записями;

- перед загрузкой данных в существующую базу следует оценить риск дублирования или конфликтов идентификаторов.

Результат изучения: умение подготовить выгрузку данных и загрузить fixture в совместимую базу данных.

4.4. Результаты изучения раздела

После выполнения заданий Яндекс Практикума необходимо уметь:

- объяснить различия между основными типами связей реляционной базы данных;
- описать таблицы средствами моделей Django;
- определить типы полей и параметры связей;
- объяснить назначение миграций и порядок их применения;
- выполнять CRUD-операции через Django ORM;
- получать и фильтровать записи;
- работать со связанными объектами;
- подготовить и загрузить fixture-файл;
- применять изученные возможности ORM в индивидуальном проекте.

Учебные задания Практикума являются подготовкой к выполнению индивидуального проекта. Их успешное завершение не заменяет реализацию и проверку функциональности собственного приложения.

5. ИНДИВИДУАЛЬНЫЙ СКВОЗНОЙ ПРОЕКТ

5.1. Общие требования к проекту

В рамках ПР9 необходимо продолжить разработку индивидуального проекта, созданного и расширенного в предыдущих практических работах. Проект не создаётся заново: сохраняются предметная область, пользовательские сценарии, структура HTML-шаблонов и реализованная функциональность.

Основная задача – заменить выполнение SQL-запросов непосредственно в представлениях Django на работу с базой данных посредством Django ORM.

В ПР7 для проекта Roomly были реализованы:

- реляционная структура базы данных SQLite;
- таблицы `rooms`, `users` и `bookings`;
- первичные и внешние ключи;
- импорт исходных данных из JSON-файлов;
- SQL-запросы для получения, фильтрации, группировки и объединения данных;
- поиск свободных помещений на заданную дату;
- статистика бронирований;
- подключение SQLite через `sqlite3` в представлениях Django;
- передача результатов запросов в HTML-шаблоны.

В ПР9 необходимо сохранить перечисленные пользовательские сценарии, изменив способ получения и обработки данных. Работа строится по этапам, описанным в подразделах 5.3–5.17: анализ существующей базы, выбор стратегии перехода, создание моделей и миграций, перенос данных, реализация операций ORM, адаптация представлений, проверка и актуализация документации проекта.

Примечание. Все этапы раздела 5 описаны на примере учебного проекта Roomly. Студенты выполняют те же преобразования на собственном индивидуальном сквозном проекте по выбранной ранее предметной области: имена моделей, приложений, файлов, таблиц и полей заменяются на сущности своего проекта, а технологическая последовательность шагов (анализ базы, модели, миграции, перенос данных, ORM-запросы, адаптация представлений) сохраняется. Команды, пути и примеры кода необходимо адаптировать к фактической структуре своего приложения.

5.2. Архитектура создаваемого проекта

Прежде чем описывать модели и переносить данные, зафиксируем архитектуру, которую получает проект в результате перехода на Django ORM. Структура проекта, сложившаяся в ПР5–ПР7, сохраняется: это веб-приложение Django с набором приложений, маршрутов и HTML-шаблонов. Изменения затрагивают слой работы с данными: вместо прямых SQL-запросов через библиотеку sqlite3 данные описываются моделями и получаются через ORM. Схема работы с данными после перехода показана на рис. 2, а сравнение подходов к доступу к данным – в таблице 4.

Веб-приложение Django строится по шаблону MTV (Model–Template–View), который определяет ответственность каждой части проекта:

- модель (Model) – классы в файлах `models.py`, описывающие сущности предметной области и выполняющие операции с базой данных через ORM;
- шаблон (Template) – HTML-шаблоны, определяющие, какие данные и в каком виде отображаются на странице;
- представление (View) – функции в файлах `views.py`, принимающие HTTP-запрос, получающие данные через модели и передающие их в шаблоны.

Связующее звено – маршрутизация: файл `urls.py` каждого приложения и корневой `urls.py` проекта связывают адреса страниц с функциями-представлениями. По каждой сущности предметной области выделяется отдельное Django-приложение: в проекте Roomly приложение `rooms` отвечает за помещения, `users` – за пользователей, `bookings` – за бронирования. В индивидуальном проекте приложения соответствуют сущностям собственной предметной области. Компоненты создаваемого проекта и потоки данных между ними показаны на рис. 3.

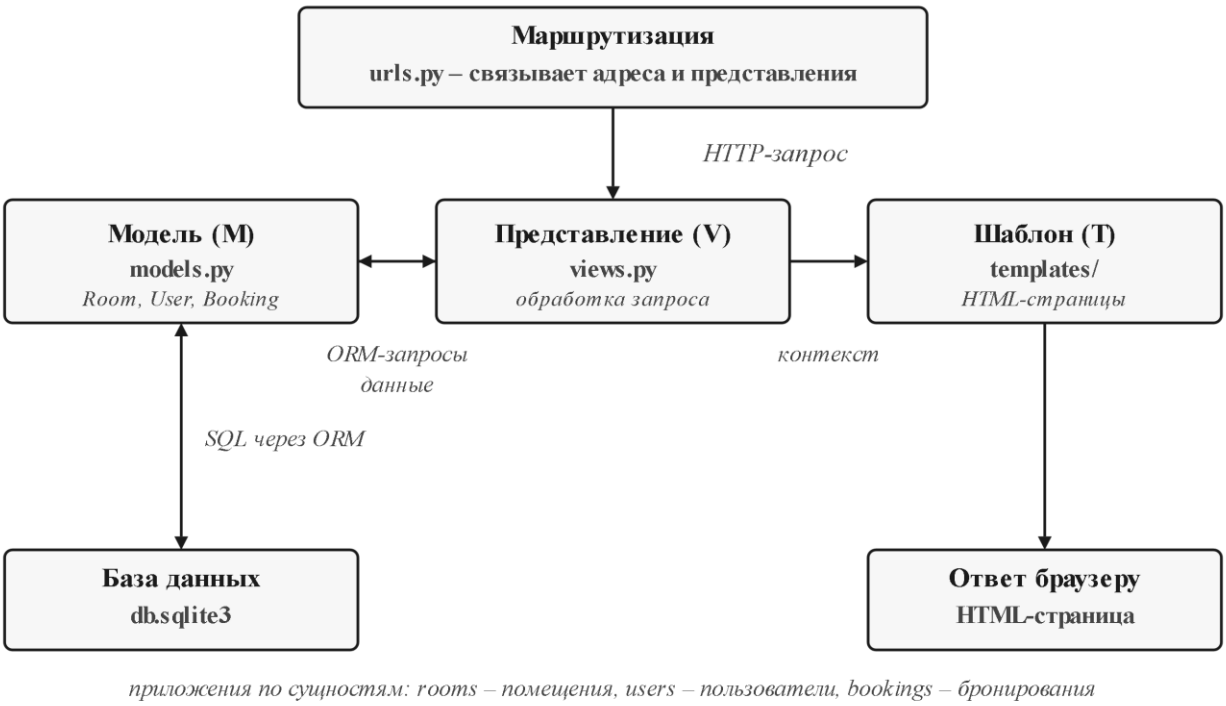


Рисунок 3 – Архитектура создаваемого проекта (шаблон MTV)

Обработка запроса пользователя в полученной архитектуре проходит следующие шаги:

1. Браузер отправляет запрос на адрес страницы, например `/rooms/` – список помещений.
2. Маршрутизация определяет по `urls.py` функцию-представление, которая должна обработать запрос.
3. Представление обращается к менеджеру модели, например `Room.objects.all()`: ORM формирует SQL-запрос к базе данных.
4. SQLite возвращает данные, ORM преобразует их в объекты моделей.
5. Представление передаёт полученные объекты в шаблон через контекст.
6. Шаблон формирует HTML-страницу, которая возвращается браузеру.

Таким образом, после перехода на ORM в проекте выделяются три слоя: слой данных – модели и миграции (подразделы 5.5 и 5.7); слой логики – представления с ORM-запросами (подразделы 5.9–5.15); слой представления – HTML-шаблоны, унаследованные из ПР5–ПР6. Представления и шаблоны при переходе сохраняются: меняется только источник данных, с которым работает код представлений.

Каталоги и файлы проекта Roomly после перехода на ORM показаны ниже. Комментарии указывают назначение каждого файла:

```
roomly/                                 # корневой каталог проекта
├── manage.py                           # служебная утилита Django
├── db.sqlite3                          # целевая база данных, с которой работает Django
├── README.md                           # описание проекта (актуализируется в 5.17)
├── roomly/                             # пакет настроек проекта
│   ├── settings.py                     # INSTALLED_APPS, TEMPLATES, DATABASES
│   └── urls.py                         # корневые маршруты
├── templates/                          # шаблоны уровня проекта
│   ├── base.html                       # базовый шаблон всех страниц
│   └── includes/
│       └── navigation.html             # общая навигация сайта (ПР6)
├── homepage/                           # приложение «Главная страница»
│   ├── views.py                        # главная страница проекта
│   ├── urls.py                         # маршруты главной страницы
│   ├── templates/
│   │   └── homepage/
│   │       └── index.html              # главная страница (ПР6)
│   └── static/
│       └── homepage/
│           ├── css/
│           │   └── style.css           # собственные стили проекта (ПР6)
│           ├── js/
│           │   └── main.js             # год в подвале страницы (ПР6)
│           └── img/
│               └── logo.png            # логотип проекта (ПР6)
├── rooms/                              # приложение «Помещения»
│   ├── models.py                       # модель Room (5.5.1)
│   ├── views.py                        # представления помещений
│   ├── urls.py                         # маршруты помещений
│   ├── migrations/                     # миграции модели Room
│   │   └── 0001_initial.py             # создание таблицы помещения
│   ├── templates/
│   │   └── rooms/
│   │       ├── room_list.html          # список помещений
│   │       ├── room_detail.html        # карточка помещения
│   │       ├── available.html          # поиск свободных помещений (5.11)
│   │       └── includes/
│   │           └── room_card.html      # карточка помещения в списке (ПР6)
├── users/                              # приложение «Пользователи»
│   ├── models.py                       # модель User (5.5.2)
│   ├── migrations/                     # миграции модели User
│   │   └── 0001_initial.py             # создание таблицы пользователя
├── bookings/                           # приложение «Бронирования»
│   ├── models.py                       # модель Booking (5.5.3)
│   ├── views.py                        # представления бронирований
│   ├── urls.py                         # маршруты бронирований
│   ├── migrations/                     # миграции модели Booking
│   │   └── 0001_initial.py             # создание таблицы бронирования
│   ├── templates/
│   │   └── bookings/
│   │       ├── booking_list.html       # список бронирований
│   │       ├── booking_detail.html     # карточка бронирования
│   │       ├── statistics.html         # страница статистики (5.12)
│   │       └── includes/
│   │           └── booking_status.html # фрагмент статуса бронирования
├── db/                                 # скрипты работы с базой данных
│   ├── database.py                     # SQL-слой ПР7, остаётся в проекте
│   ├── inspect_source_db.py            # инспекция исходной базы (5.8.1)
│   └── import_sqlite_to_orm.py         # перенос данных в ORM (5.8.2)
├── sql/                                # SQL-слой ПР7, остаётся в проекте
│   ├── schema.sql                      # схема таблиц предыдущей реализации
│   └── queries.sql                     # запросы предыдущей реализации
├── data/                               # исходные данные предыдущих работ
│   ├── roomly.sqlite                   # исходная база данных ПР7
│   ├── rooms.json                      # помещения (данные ранних работ)
│   ├── users.json                      # пользователи (данные ранних работ)
│   └── bookings.json                   # бронирования (данные ранних работ)
```

Примечание. Структура приложений и имена файлов показаны на примере проекта Roomly. В индивидуальном проекте состав приложений определяется перечнем сущностей, полученным на этапе анализа базы данных (подраздел 5.3), а количество моделей может отличаться от примера. Технологическая последовательность шагов при этом сохраняется.

5.3. Анализ существующей базы данных

До изменения кода необходимо изучить текущую реализацию проекта. В проекте Roomly используются три основные сущности (таблица 5).

Сущность	Основные поля	Назначение
Room – помещение	`id`, `name`, `capacity`	хранение информации о помещениях
User – пользователь	`id`, `name`, `email`	хранение информации о пользователях системы
Booking – бронирование	`id`, `room_id`, `user_id`, `booking_date`, `is_cancelled`	хранение сведений о бронировании помещений

Связи между сущностями:

- одно помещение может иметь несколько бронирований;
- один пользователь может создавать несколько бронирований;
- каждое бронирование относится к одному помещению и одному пользователю.

Структура связей показана на рис. 4. Обозначения 1 и N соответствуют связям «один ко многим»; внешние ключи находятся в таблице `bookings`.

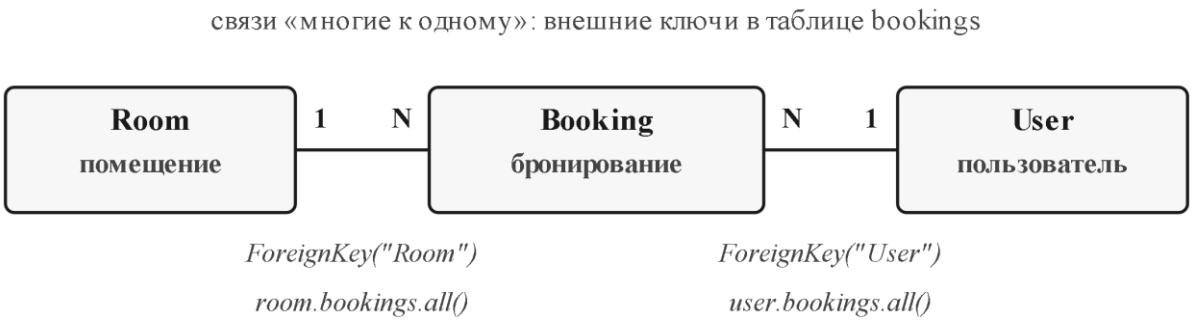


Рисунок 4 – Связи сущностей индивидуального проекта

Перед началом изменений необходимо:

- открыть проект в используемой среде разработки;
- найти настройки Django в `settings.py`;
- определить, какая база данных указана в параметре `DATABASES`;
- найти файл `data/roomly.sqlite`, если он используется в проекте;
- проверить структуру таблиц и существующие данные;
- найти представления, в которых используются `sqlite3.connect()`, `cursor.execute()` и другие операции с прямыми SQL-запросами;
- создать резервную копию существующей базы данных.

Проверьте конфигурацию Django. В терминале из корневого каталога проекта выполните:

```
# Проверяем конфигурацию Django
python manage.py check
```

Команда `check` проверяет настройки Django, регистрацию приложений и некоторые другие параметры проекта. Затем определите назначение файлов баз данных: в проекте `Roomly` их два – `data/roomly.sqlite` (исходная база данных, созданная в ходе предыдущих практических работ) и db.sqlite3 (целевая база данных, с которой работает Django).

В файле `roomly/settings.py` подключение к целевой базе настроено следующим образом:

```
DATABASES = {
    'default': {
        # Используется база данных SQLite
        'ENGINE': 'django.db.backends.sqlite3',

        # Путь к базе относительно корня проекта
        'NAME': BASE_DIR / 'db.sqlite3',
    }
}
```

Параметр `NAME` определяет базу данных, используемую Django по умолчанию. Поэтому команды `migrate`, `makemigrations` и запросы через Django ORM не будут автоматически обращаться к data/roomly.sqlite. В рамках ПР9 данные переносятся из исходной базы в целевую, после чего представления получают и изменяют данные через ORM, работающую с db.sqlite3. Не меняйте настройку `DATABASES` на `data/roomly.sqlite` ради упрощения переноса: исходный файл должен использоваться как источник, а целевой – как база Django.

Создайте резервную копию исходной базы. В терминале PowerShell выполните:

```
# Создаём резервную копию исходной базы
Copy-Item .\data\roomly.sqlite .\data\roomly_backup.sqlite

# Проверяем наличие исходной базы и резервной копии
Get-Item .\data\roomly.sqlite
Get-Item .\data\roomly_backup.sqlite
```

Резервная копия понадобится для восстановления исходных данных в случае ошибки при дальнейшей работе. Дополнительно выполните глобальный поиск по проекту (в Visual Studio Code – Ctrl+Shift+F) по выражениям `sqlite3.connect`, `cursor.execute` и `sqlite3`: для каждого найденного фрагмента определите, к какой функции и странице он относится. Запишите названия соответствующих файлов и функций – это позволит заменять SQL-запросы последовательно, не нарушая работу

существующих страниц. На этом этапе не удаляйте SQL-запросы и не переписывайте шаблоны: сначала подготавливаются модели и целевая база данных.

Результатом этапа должен стать перечень таблиц, полей, связей и представлений, которые необходимо адаптировать.

Примечание. Состав сущностей, таблиц и связей определяется предметной областью индивидуального проекта студента и может отличаться от примера Roomly. Перечень, составленный на этом этапе, задаёт план дальнейшей работы: каждой сущности исходной базы будет соответствовать модель Django.

Важно. Существующая база данных содержит результаты предыдущей работы. Нельзя без предварительной проверки удалять её, пересоздавать таблицы или применять миграции к ней в расчёте на то, что Django автоматически распознает существующую структуру.

5.4. Выбор стратегии перехода к Django ORM

Перед созданием моделей необходимо определить, как будут использоваться данные, сохранённые в ПР7. Возможны два подхода.

Вариант 1. Подключение ORM к существующим таблицам. Модели Django настраиваются для работы с уже существующими таблицами посредством параметра `Meta.db_table`. При необходимости задаются соответствия между именами полей моделей и столбцами таблиц с помощью `db_column`.

Этот подход позволяет сохранить существующие таблицы, но требует точного соответствия между их структурой и описанием моделей. Также необходимо отдельно учитывать состояние миграций Django: наличие таблиц само по себе не означает, что Django зарегистрировал их создание в истории миграций. Применять этот вариант можно только после проверки структуры таблиц и плана миграций: нельзя без проверки выполнять обычную миграцию, которая попытается создать уже существующие таблицы.

Вариант 2. Создание таблиц, управляемых Django, и перенос данных. Существующая база данных ПР7 сохраняется как источник исходных данных. Django использует базу, указанную в настройках `DATABASES`, и создаёт в ней таблицы для ORM-моделей. Затем данные из прежней базы переносятся в новые таблицы.

Этот подход позволяет не изменять исходную базу данных и независимо проверить результат переноса. При этом необходимо сохранить идентификаторы записей и связи между ними.

Для учебного проекта рекомендуется использовать второй вариант, если ранее созданные таблицы не соответствуют стандартной структуре таблиц Django или состояние миграций неизвестно. Далее в методических указаниях рассматривается именно этот вариант. Если в проекте уже настроено управление существующими таблицами через Django ORM, следует сохранить согласованную конфигурацию и адаптировать дальнейшие шаги к ней.

5.5. Создание моделей Django

Модель Django – это класс, описывающий структуру данных и предоставляющий интерфейс для работы с записями базы данных. Модели должны находиться в `models.py` соответствующих Django-приложений. Их не следует путать с обычными Python-классами предметной области, которые могли быть созданы в предыдущих практических работах.

Примечание. В подразделах 5.5.1–5.5.3 модели показаны на примере Roomly. В индивидуальном проекте создаются модели по сущностям собственной предметной области: количество моделей, их имена, поля и связи определяются составом таблиц, выявленным на этапе анализа (подраздел 5.3).

В проекте Roomly целесообразно разместить:

- модель `Room` в `rooms/models.py`;
- модель `User` в `users/models.py`;
- модель `Booking` в `bookings/models.py`.

Если приложение `users` ещё не создано, его можно добавить командой `python manage.py startapp users`. После этого приложение необходимо зарегистрировать в `INSTALLED_APPS` файла `settings.py`.

Примечание. Модель User в данном примере представляет пользователя предметной области Roomly. Она не является встроенной моделью аутентификации Django и сама по себе не обеспечивает регистрацию или вход в систему.

5.5.1. Модель помещения Room

В файле `rooms/models.py` необходимо описать модель помещения:

```
from django.db import models

class Room(models.Model):
    # Название помещения.
    # max_length задаёт максимально допустимую длину строки.
    name = models.CharField(
        max_length=200,
        verbose_name="Название помещения",
    )

    # Вместимость помещения в количестве человек.
    # Поле предназначено для неотрицательных целых чисел.
    capacity = models.PositiveIntegerField(
        verbose_name="Вместимость",
    )

    def __str__(self):
        # Текстовое представление объекта в административной
        # панели, оболочке Django и других местах вывода.
        return self.name
```

```
    class Meta:
        # Сортировка объектов по названию по умолчанию.
        ordering = ["name"]

        # Человекочитаемые названия модели.
        verbose_name = "Помещение"
        verbose_name_plural = "Помещения"
```

В модели определены следующие поля:

- `name` – название помещения;
- `capacity` – вместимость помещения.

Поле `id` явно не указано: Django автоматически создаёт первичный ключ, если он не объявлен в модели самостоятельно. Метод `__str__()` возвращает название помещения – это упрощает отображение объекта в административном интерфейсе и при работе через Django shell.

В модели используются следующие элементы Django ORM:

- `models.Model` – базовый класс моделей Django;
- `CharField` – поле для строк ограниченной длины;
- `PositiveIntegerField` – поле для неотрицательных целых чисел;
- `__str__()` – метод, задающий текстовое представление объекта;
- `Meta.ordering` – сортировка записей по умолчанию;
- `verbose_name` и `verbose_name_plural` – человекочитаемые названия модели.

Если в предыдущей версии проекта существовал метод `is_suitable_for()`, его можно сохранить как метод модели:

```
def is_suitable_for(self, people_count):
    return self.capacity >= people_count
```

Тогда модель будет содержать и данные, и относящееся к помещению поведение.

5.5.2. Модель пользователя User

В файле `users/models.py` необходимо описать пользователя предметной области:

```
from django.db import models

class User(models.Model):
    # Имя пользователя системы бронирования.
    name = models.CharField(
        max_length=200,
        verbose_name="Имя пользователя",
```

```
    )

    # Электронная почта пользователя.
    # EmailField предназначен для хранения адресов электронной почты.
    email = models.EmailField(
        verbose_name="Электронная почта",
    )

    def __str__(self):
        # Возвращаем имя пользователя при выводе объекта.
        return self.name

    class Meta:
        # Сортировка пользователей по имени.
        ordering = ["name"]

        # Человекочитаемые названия модели.
        verbose_name = "Пользователь"
        verbose_name_plural = "Пользователи"
```

Поля модели:

- `name` – имя пользователя;
- `email` – адрес электронной почты.

`EmailField` предназначен для хранения адресов электронной почты. Он предоставляет стандартные возможности проверки формата, но сам по себе не гарантирует уникальность адреса. Если по правилам проекта адрес должен быть уникальным, это ограничение необходимо задать отдельно. Поле id также создаётся Django автоматически. Класс `Meta`, как и в модели `Room`, задаёт сортировку объектов по имени и человекочитаемые названия модели.

5.5.3. Модель бронирования Booking

В файле `bookings/models.py` необходимо описать бронирование:

```
from django.db import models

# Импортируем модели, с которыми будет связано бронирование.
from rooms.models import Room
from users.models import User

class Booking(models.Model):
    # Помещение, для которого оформлено бронирование.
    # Внешний ключ связывает Booking с Room.
    room = models.ForeignKey(
        Room,
        on_delete=models.PROTECT,
```

```
        related_name="bookings",
        verbose_name="Помещение",
    )

    # Пользователь, оформивший бронирование.
    # Внешний ключ связывает Booking с User.
    user = models.ForeignKey(
        User,
        on_delete=models.PROTECT,
        related_name="bookings",
        verbose_name="Пользователь",
    )

    # Дата бронирования.
    # DateField хранит дату, а не произвольную строку.
    booking_date = models.DateField(
        verbose_name="Дата бронирования",
    )

    # Признак отмены бронирования.
    # Новое бронирование по умолчанию считается активным.
    is_cancelled = models.BooleanField(
        default=False,
        verbose_name="Бронирование отменено",
    )

    def cancel(self):
        # Помечаем бронирование как отменённое.
        self.is_cancelled = True

        # Сохраняем только изменённое поле.
        self.save(update_fields=["is_cancelled"])

    def __str__(self):
        # Формируем строковое представление бронирования.
        return (
            f"{self.room} — {self.booking_date} "
            f"({self.user})"
        )

    class Meta:
        # Сначала сортируем по дате, затем по идентификатору.
        ordering = ["booking_date", "pk"]

        # Человекочитаемые названия модели.
        verbose_name = "Бронирование"
```

```
        verbose_name_plural = "Бронирования"
```

В модели используются следующие поля:

- `room` – внешний ключ на помещение;
- `user` – внешний ключ на пользователя;
- `booking_date` – дата бронирования (DateField, формат YYYY-MM-DD);
- `is_cancelled` – признак отмены бронирования.

В отличие от исходной таблицы, где хранились `room_id` и `user_id`, в коде Django используются поля `room` и `user`. При этом Django автоматически создаёт соответствующие столбцы `room_id` и `user_id`.

Поле `booking_date` описано как `DateField`: Django хранит в нём дату, а не произвольную строку, и при фильтрации сравнивает значение с объектом `date`. Это обеспечивает корректные сравнения дат в запросах (подразделы 5.11 и 5.13). При переносе данных строки исходной базы формата `YYYY-MM-DD` преобразуются в объекты `date` (подраздел 5.8.2).

Параметр `on_delete=models.PROTECT` запрещает удаление помещения или пользователя, если на них ссылаются бронирования: это позволяет сохранить историю бронирований. Выбор стратегии удаления должен соответствовать правилам предметной области.

Параметр `related_name="bookings"` задаёт имя обратного доступа и позволяет получить связанные бронирования через обратную связь:

```
room.bookings.all()
user.bookings.all()
```

Метод `cancel()` отменяет бронирование и сохраняет изменение в базе данных, а вызов `save(update_fields=["is_cancelled"])` обновляет только указанное поле.

Метод `__str__()` возвращает строковое представление с помещением, датой и пользователем – оно используется в административном интерфейсе и оболочке Django. Класс `Meta` задаёт сортировку по дате и идентификатору, а также человекочитаемые названия модели.

При использовании моделей необходимо также определить, где будет проверяться правило доступности помещения. Одного описания внешних ключей недостаточно для запрета двух активных бронирований одного помещения на одну дату: эта проверка должна быть реализована в логике создания бронирования и протестирована отдельно. В данной работе такая проверка не реализуется: операции создания записей через веб-интерфейс (формы) рассматриваются в последующих практиче-

ских работах. Отдельное ограничение уникальности для сочетания помещения и даты на данном этапе не добавляется: оно потребовало бы дополнительного согласования с правилами проекта, в частности с учётом отменённых бронирований.

5.6. Проверка регистрации приложений и моделей

После создания моделей необходимо проверить регистрацию приложений и корректность импортов. В `settings.py` в список `INSTALLED_APPS` должны входить приложения, содержащие модели:

```
INSTALLED_APPS = [
    # Встроенные приложения Django
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',

    # Приложения проекта Roomly
    'homepage',
    'rooms',
    'bookings',
    'users',
]
```

Если приложение уже зарегистрировано, повторно добавлять его не нужно.

Далее необходимо выполнить проверку конфигурации:

```
python manage.py check
```

Команда проверяет основные настройки Django. При возникновении ошибок необходимо исправить импорты, конфигурацию приложений и определения моделей до создания миграций.

5.7. Создание и применение миграций

Миграции описывают изменения структуры базы данных и позволяют Django согласованно управлять таблицами. Создание моделей в Python ещё не создаёт соответствующие таблицы в базе данных: необходимо сформировать и применить миграции.

Шаг 1. Создание файлов миграций. Выполнить команду:

```
python manage.py makemigrations
```

Django проанализирует изменения в моделях и создаст файлы миграций для зарегистрированных приложений. При необходимости можно указать приложения явно:

```
python manage.py makemigrations rooms users bookings
```

Шаг 2. Проверка миграций. Проверить, какие миграции ожидают применения:

```
python manage.py showmigrations
```

Для изучения SQL, который будет использован при выполнении конкретной миграции, можно применить команду `python manage.py sqlmigrate rooms 0001`. Вместо `0001` необходимо указать фактический номер миграции.

Шаг 3. Применение миграций. Выполнить:

```
python manage.py migrate
```

После выполнения команды Django создаст необходимые таблицы в настроенной базе данных и зарегистрирует применённые миграции.

Повторно проверьте состояние миграций и конфигурацию проекта:

```
# Проверяем, что миграции применены
python manage.py showmigrations

# Выполняем контрольную проверку проекта
python manage.py check
```

У применённых миграций отображается отметка [X].

Важно. Перед применением миграций необходимо убедиться, что Django работает с ожидаемой базой данных. Если прежние таблицы находятся в отдельном файле data/roomly.sqlite, а в DATABASES указана другая база, миграции будут применяться к базе, настроенной в Django, а не автоматически к исходному файлу. Если Django должен работать непосредственно с прежними таблицами, обычного выполнения makemigrations и migrate недостаточно: предварительно необходимо согласовать существующую структуру таблиц с моделями и историей миграций.

Файлы миграций необходимо сохранять в системе контроля версий: это часть исходного кода проекта, позволяющая воспроизвести структуру базы данных в другом окружении.

5.8. Перенос данных из базы ПР7

Если выбрана стратегия создания таблиц, управляемых Django, исходную базу необходимо сохранить без изменений и перенести данные в новые таблицы. Для Roomly требуется перенести:

- помещения;
- пользователей;
- бронирования со ссылками на соответствующие помещения и пользователи.

Перенос следует выполнять именно в таком порядке, поскольку бронирования зависят от двух других сущностей.

Примечание. Порядок переноса повторяет зависимости сущностей индивидуального проекта: сначала переносятся независимые сущности, затем те, которые ссылаются на них внешними

ключами. Имена исходных таблиц, столбцов и файлов баз данных определяются по результатам инспекции конкретного проекта (подраздел 5.8.1).

5.8.1. Подготовка к переносу

Перед переносом необходимо:

- убедиться, что исходная база доступна;
- проверить названия столбцов;
- проверить наличие и уникальность идентификаторов;
- убедиться, что все `room_id` и `user_id` в бронированиях указывают на существующие записи;
- проверить значения `is_cancelled`;
- сохранить резервную копию исходной базы.

Фактическую структуру исходной базы удобно исследовать отдельным скриптом, который выведет названия таблиц, сведения о столбцах и количество записей. Создайте в каталоге db файл `db/inspect_source_db.py`:

```
import sqlite3
from pathlib import Path

# Определяем корневой каталог проекта.
# Файл скрипта находится в подкаталоге db,
# поэтому родительский каталог на уровень выше — корень Roomly.
BASE_DIR = Path(__file__).resolve().parent.parent

# Указываем путь к исходной базе, созданной на предыдущих работах.
SOURCE_DB = BASE_DIR / "data" / "roomly.sqlite"

# Проверяем, что файл исходной базы существует.
if not SOURCE_DB.exists():
    raise FileNotFoundError(
        f"Исходная база не найдена: {SOURCE_DB}"
    )

# Подключаемся к исходной базе SQLite.
# Скрипт выполняет только запросы на чтение.
with sqlite3.connect(SOURCE_DB) as connection:
    # Позволяем обращаться к значениям строк по именам столбцов.
    connection.row_factory = sqlite3.Row

    # Получаем список пользовательских таблиц базы данных.
    tables = connection.execute(
        """
        SELECT name
        FROM sqlite_master
```

```
        WHERE type = 'table'
          AND name NOT LIKE 'sqlite_%'
        ORDER BY name
        """
    ).fetchall()

    # Последовательно исследуем каждую таблицу.
    for (table_name,) in tables:
        print(f"\nТаблица: {table_name}")

        # Получаем описание столбцов таблицы:
        # имена, типы, признаки NULL, ключи и значения по умолчанию.
        columns = connection.execute(
            f'PRAGMA table_info("{table_name}")'
        ).fetchall()

        print("Столбцы:")
        for column in columns:
            print(
                f" имя={column['name']}, "
                f"тип={column['type']}, "
                f"NOT NULL={column['notnull']}, "
                f"PRIMARY KEY={column['pk']}"
            )

        # Подсчитываем количество записей в таблице.
        count = connection.execute(
            f'SELECT COUNT(*) FROM "{table_name}"'
        ).fetchone()[0]

        print(f"Количество записей: {count}")
```

Запустите скрипт из корневого каталога проекта:

```
# Исследуем структуру исходной базы
python db/inspect_source_db.py
```

Скрипт выводит имена пользовательских таблиц, сведения о столбцах (имена, типы, признак `NOT NULL`, первичный ключ) и количество записей. Запросы выполняются только на чтение: исходная база не изменяется. По результатам выполнения установите, какие таблицы соответствуют сущностям проекта, и сверьте фактические названия столбцов с ожидаемыми.

Особое внимание уделите таблице бронирований: `room_id` должен ссылаться на существующие записи помещений, `user_id` — на пользователей; значения `booking_date` должны быть датами формата `YYYY-MM-DD`, а `is_cancelled` — кор-

ректно различать активные (0) и отменённые (1) записи. Если инспекция показывает другой формат, обработку значений необходимо изменить до запуска скрипта переноса.

Примечание. В примерах этого подраздела предполагается, что исходные таблицы имеют названия rooms, users и bookings, а база находится в файле data/roomly.sqlite. Если фактическая структура отличается, названия таблиц и столбцов необходимо адаптировать.

5.8.2. Реализация переноса

Для переноса можно создать отдельный скрипт `db/import_sqlite_to_orm.py`, который прочитает данные из прежней базы и создаст записи с помощью ORM. Пример скрипта:

```
import os
import sys
import sqlite3
from datetime import date
from pathlib import Path

import django
from django.db import transaction

# Определяем корневой каталог проекта Roomly.
# Скрипт находится в каталоге db, поэтому поднимаемся на уровень выше.
BASE_DIR = Path(__file__).resolve().parent.parent

# Добавляем корень проекта в пути поиска модулей Python.
# Это позволяет найти пакет настроек при запуске скрипта из каталога db.
sys.path.insert(0, str(BASE_DIR))

# Указываем путь к исходной базе данных.
SOURCE_DB = BASE_DIR / "data" / "roomly.sqlite"

# Указываем модуль настроек Django.
os.environ.setdefault(
    "DJANGO_SETTINGS_MODULE",
    "roomly.settings",
)

# Инициализируем Django после настройки пути и переменной окружения.
django.setup()

# Импортируем модели только после инициализации Django.
from rooms.models import Room
from users.models import User
from bookings.models import Booking


def main():
    # Проверяем, что исходная база существует.
    if not SOURCE_DB.exists():
        raise FileNotFoundError(
            f"Исходная база не найдена: {SOURCE_DB}"
        )
```

```
# Открываем исходную базу для чтения.
# row_factory позволяет обращаться к полям по имени.
with sqlite3.connect(SOURCE_DB) as connection:
    connection.row_factory = sqlite3.Row

    # Считываем все помещения, сохраняя их идентификаторы.
    rooms = connection.execute(
        """
        SELECT id, name, capacity
        FROM rooms
        ORDER BY id
        """
    ).fetchall()

    # Считываем пользователей и их идентификаторы.
    users = connection.execute(
        """
        SELECT id, name, email
        FROM users
        ORDER BY id
        """
    ).fetchall()

    # Считываем бронирования вместе с внешними ключами,
    # датами и признаками отмены.
    bookings = connection.execute(
        """
        SELECT
            id,
            room_id,
            user_id,
            booking_date,
            is_cancelled
        FROM bookings
        ORDER BY id
        """
    ).fetchall()

    # Выполняем запись в целевую базу одной транзакцией.
    # При ошибке изменения внутри транзакции откатываются.
    with transaction.atomic():

        # Сначала переносим помещения, так как на них
        # будут ссылаться записи бронирований.
        for row in rooms:
            Room.objects.update_or_create(
                # Сохраняем исходный первичный ключ.
                pk=row["id"],

                # Значения полей создаваемой или обновляемой записи.
                defaults={
                    "name": row["name"],
                    "capacity": row["capacity"],
                },
```

```
                )

            # Затем переносим пользователей.
            for row in users:
                User.objects.update_or_create(
                    # Сохраняем исходный идентификатор пользователя.
                    pk=row["id"],
                    defaults={
                        "name": row["name"],
                        "email": row["email"],
                    },
                )

            # После помещений и пользователей переносим бронирования.
            for row in bookings:
                # Преобразуем строку ISO-формата в объект date,
                # необходимый для поля Django DateField.
                booking_date = date.fromisoformat(
                    row["booking_date"]
                )

                Booking.objects.update_or_create(
                    # Сохраняем исходный идентификатор бронирования.
                    pk=row["id"],
                    defaults={
                        # Внешние ключи ссылаются на перенесённые записи.
                        "room_id": row["room_id"],
                        "user_id": row["user_id"],

                        # Сохраняем дату и состояние бронирования.
                        "booking_date": booking_date,
                        "is_cancelled": bool(
                            row["is_cancelled"]
                        ),
                    },
                )

    # Выводим количество записей в целевой базе.
    # Эти значения необходимо сопоставить с результатами инспекции.
    print("Перенос данных завершён.")
    print(f"Комнат: {Room.objects.count()}")
    print(f"Пользователей: {User.objects.count()}")
    print(f"Бронирований: {Booking.objects.count()}")


if __name__ == "__main__":
    # Запускаем перенос только при непосредственном
    # выполнении этого файла.
    main()
```

Скрипт предполагает, что файл `data/roomly.sqlite` является отдельной исходной базой, а Django настроен на базу `db.sqlite3`, в которой уже применены миграции. Если фактические названия таблиц, столбцов или модуля настроек отличаются, скорректируйте SQL-запросы и значения в скрипте до запуска.

Метод `update_or_create()` ищет запись по первичному ключу: если запись с таким идентификатором существует, она обновляется, если её нет – создаётся новая. Это удобно для повторного запуска переноса, но не заменяет проверку результата.

Порядок переноса имеет значение: сначала создаются помещения и пользователи, а затем бронирования, ссылающиеся на них через внешние ключи.

Конструкция `transaction.atomic()` объединяет операции записи в транзакцию: при исключении внутри транзакции выполненные в ней изменения откатываются, поэтому частичный перенос не сохранится.

Строки `booking_date` из исходной базы преобразуются в объекты `date` методом `date.fromisoformat()`: поле модели `Booking` описано как `DateField` и ожидает объект даты, а не строку.

Скрипт не удаляет записи целевой базы, которых нет в исходной: повторный импорт обновит совпадающие по идентификаторам записи, но не удалит посторонние.

Для запуска скрипта из корневого каталога проекта:

```
# Переносим данные из исходной базы в целевую базу Django
python db/import_sqlite_to_orm.py
```

Выполнять перенос необходимо после применения миграций. При успешном выполнении в терминале появится сообщение о завершении переноса и количество помещений, пользователей и бронирований в целевой базе. Эти количества должны быть сопоставлены с результатами инспекции исходной базы.

5.8.3. Проверка перенесённых данных

После переноса необходимо сравнить количество записей в исходной и новой базах. Проверить можно через Django shell:

```
python manage.py shell
```

В интерактивной оболочке выполнить:

```
# Импортируем модели, созданные для Roomly.
from rooms.models import Room
from users.models import User
from bookings.models import Booking

# Проверяем количество записей в целевой базе.
print("Комнаты:", Room.objects.count())
print("Пользователи:", User.objects.count())
print("Бронирования:", Booking.objects.count())
```

Количество записей должно соответствовать исходным данным. Для примера из предыдущей работы ожидаются три помещения, один пользователь и два бронирования, если данные не изменялись.

Затем выведите бронирования вместе со связанными помещениями и пользователями – это одновременно проверит внешние ключи:

```
# Получаем бронирования и связанные объекты одним запросом
# с использованием select_related().
bookings = Booking.objects.select_related(
    "room",
    "user",
).order_by("pk")

# Проверяем идентификаторы, связи, даты и признаки отмены.
for booking in bookings:
    print(
        booking.pk,
        booking.room.name,
        booking.user.name,
        booking.booking_date,
        booking.is_cancelled,
    )
```

Сопоставьте результат с исходной базой: идентификаторы сохранены, бронирования относятся к правильным помещениям и пользователям, даты перенесены без искажений, отменённые записи сохранили свой статус. Выйдите из оболочки командой `exit()`.

Примечание. Для просмотра и редактирования записей базы данных удобно установить расширение SQLite Explorer в VS Code (Marketplace). Открыв через него файл db.sqlite3, можно просматривать таблицы, изменять значения полей и добавлять записи, необходимые для проверки работы приложения.

Если перенос завершился с ошибкой, следует исправить причину и повторить его, не удаляя исходную базу.

5.9. Реализация операций с данными через ORM

После создания моделей и переноса данных необходимо освоить основные операции ORM на сущностях Roomly. Операции можно проверять в Django shell и затем переносить в представления проекта.

Подразделы 5.9.1 и 5.9.2 показывают операции чтения данных средствами ORM. Операции добавления, изменения и удаления записей разобраны в подразделе 3.5 и отрабатывались в Django shell при выполнении Практикума (подраздел 4.3.5); их веб-реализация с использованием форм не входит в данную работу и будет изучена позже.

5.9.1. Получение списка помещений

Получение всех помещений:

```
from rooms.models import Room
```

```
rooms = Room.objects.all()
```

Поиск помещений с вместимостью не менее 30 человек:

```
rooms = Room.objects.filter(capacity__gte=30)
```

Сортировка по названию и обратная сортировка по вместимости:

```
rooms = Room.objects.order_by("name")
rooms = Room.objects.order_by("-capacity")
```

ORM формирует SQL-запрос к базе данных на основании вызовов методов. В представлении больше не требуется вручную создавать курсор и выполнять SQL-запрос.

5.9.2. Получение одного помещения

Получение помещения по идентификатору:

```
room = Room.objects.get(pk=room_id)
```

Метод `get()` ожидает ровно одну запись. Если запись не найдена или найдено несколько записей, возникает исключение.

В представлении Django обычно применяется `get_object_or_404()`:

```
from django.shortcuts import get_object_or_404
from rooms.models import Room

room = get_object_or_404(Room, pk=room_id)
```

Если помещение не существует, Django возвращает ответ 404, а не завершает обработку запроса необработанным исключением.

5.10. Использование связей между моделями

В ORM внешние ключи позволяют получать связанные объекты без написания явных JOIN в представлениях.

Получение бронирования вместе с помещением и пользователем:

```
booking = Booking.objects.select_related(
    "room",
    "user",
).get(pk=booking_id)

print(booking.room.name)
print(booking.user.name)
```

Метод `select_related()` позволяет получать связанные объекты посредством SQL-соединения, сокращая количество обращений к базе данных при чтении внешних ключей.

Это два разных подхода к получению связанных данных, и в проекте они применяются к разным задачам:

- `select_related("room", "user")` – когда нужны объекты бронирования целиком вместе со связанными объектами `room` и `user`, например в карточке бронирования;
- `annotate()` и `values()` – когда для списка или отчёта нужны только конкретные поля, например название помещения и имя пользователя.

Получение бронирований определённого помещения через обратную связь:

```
room = Room.objects.get(pk=room_id)
bookings = room.bookings.all()
```

Получение активных бронирований и активных бронирований конкретного помещения:

```
active_bookings = Booking.objects.filter(is_cancelled=False)

active_bookings = Booking.objects.filter(
    room_id=room_id,
    is_cancelled=False,
)
```

В этих примерах `room_id` – атрибут, соответствующий столбцу внешнего ключа. Также можно использовать `room=room`, если объект помещения уже получен.

5.11. Адаптация представлений помещений

После проверки моделей и операций ORM необходимо переработать представления, которые ранее обращались к базе данных через sqlite3 или читали данные из JSON-файлов. В приложении `rooms` проекта `Roomly` к ним относятся `rooms()`, `room_detail()` и `available_rooms()`: первая выполняет SQL-запрос к исходной базе, вторая загружает помещения из `data/rooms.json` и ищет запись в Python, третья выполняет SQL-запрос для поиска свободных помещений. Перевод выполняется последовательно, по одному представлению, с проверкой после каждого шага.

Перед изменением кода проверьте, что модели доступны по импортам:

```
from rooms.models import Room

from bookings.models import Booking
```

Модель `Room` используется для получения помещений, а модель `Booking` – для анализа бронирований. Не удаляйте на этом этапе исходную базу и JSON-файлы: они могут понадобиться для проверки переноса данных и работы других частей проекта. Задача состоит в замене способа получения данных, а не в немедленном удалении старых файлов.

Шаг 1. Перевод списка помещений. В файле `rooms/views.py` найдите функцию `rooms()`, выполняющую SQL-запрос к исходной базе, и замените её кодом на ORM:

```
def rooms(request):
    # Получаем все помещения из базы данных Django.
    # Сортируем их по названию.
    rooms_list = Room.objects.order_by("name")

    # Передаём список помещений в шаблон.
    return render(
        request,
        "rooms/room_list.html",
        {"rooms": rooms_list},
    )
```

Здесь `Room.objects` – менеджер модели, через который выполняются запросы; `order_by("name")` – сортировка помещений по названию; `{"rooms": rooms_list}` – контекст, передаваемый шаблону. Имя переменной в Python изменено на `rooms_list`, чтобы не затенять название модели `Room` и не путать список объектов с отдельным помещением. Имя `rooms` в контексте шаблона сохранено, поэтому менять `room_list.html` не требуется.

Проверьте маршрут в `rooms/urls.py`: имя представления в `path()` должно совпадать с фактическим именем функции в `views.py`, а имя маршрута (например, `list`) должно сохраниться, поскольку ссылки вида `{% url 'rooms:list' %}` в шаблонах зависят от имени маршрута. Запустите сервер и откройте `/rooms/`: список помещений отображается, отсортирован по названию и загружается без ошибок.

Шаг 2. Перевод карточки помещения. В текущей реализации `room_detail()` загружает список помещений из `data/rooms.json` и ищет нужное помещение функцией `find_room_by_id()`. Поскольку источник данных – модель `Room`, загружать весь список из JSON и выполнять поиск в Python больше не требуется. Замените функцию кодом с `get_object_or_404()`:

```
from django.shortcuts import get_object_or_404, render

def room_detail(request, room_id):
    # Получаем помещение по первичному ключу.
    # Если помещение не найдено, возвращаем ответ 404.
    room = get_object_or_404(
        Room,
        pk=room_id,
    )

    # Передаём объект помещения в шаблон.
```

```
    context = {
        "room": room,
    }

    return render(
        request,
        "rooms/room_detail.html",
        context,
    )
```

Функция `get_object_or_404()` из `django.shortcuts` возвращает найденный объект либо формирует ответ `404`, если помещение с заданным идентификатором отсутствует. Указывать `status=404 if room is None else 200` больше не нужно: при отсутствии помещения выполнение функции прекращается внутри `get_object_or_404()`, и Django возвращает ответ 404 автоматически. Если импорт `render` уже присутствует в начале файла, не дублируйте его – достаточно добавить `get_object_or_404` к существующему импорту.

Проверьте два сценария: страница существующего помещения отображает карточку с корректными данными, а запрос с несуществующим идентификатором (например, `/rooms/999999/`) возвращает ответ `404`. Обращения к полям в шаблоне продолжают работать через точечную нотацию:

```
{{ room.name }}

{{ room.capacity }}
```

Выражения с квадратными скобками в шаблоне Django не используются. Если шаблон использует другие атрибуты, проверьте, что они действительно определены в модели `Room`; не добавляйте новые поля модели только ради совпадения с шаблоном.

Шаг 3. Перевод поиска свободных помещений. Представление `available_rooms()` объединяет данные о помещениях и бронированиях: помещение считается свободным, если на выбранную дату у него нет активного бронирования, а отменённые бронирования помещение не блокируют. Функция получает дату из GET-параметра, проверяет её корректность и исключает помещения с активными бронированиями. Замените функцию следующим кодом:

```
def available_rooms(request):
    # Получаем дату из GET-параметра.
    booking_date = request.GET.get("date", "").strip()

    # По умолчанию список свободных помещений пуст.
    rooms_list = []
    error = None
```

```
    # Выполняем поиск только при указанной дате.
    if booking_date:
        try:
            # Преобразуем строку в объект даты.
            selected_date = datetime.strptime(
                booking_date,
                "%Y-%m-%d",
            ).date()
        except ValueError:
            # Обрабатываем некорректную дату.
            error = "Укажите корректную дату."
        else:
            # Получаем идентификаторы помещений,
            # для которых существуют активные бронирования
            # на выбранную дату.
            booked_room_ids = Booking.objects.filter(
                booking_date=selected_date,
                is_cancelled=False,
            ).values_list(
                "room_id",
                flat=True,
            )

            # Исключаем занятые помещения.
            # Оставшиеся помещения сортируем по названию.
            rooms_list = Room.objects.exclude(
                pk__in=booked_room_ids,
            ).order_by("name")

    # Передаём результат поиска в шаблон.
    context = {
        "rooms": rooms_list,
        "booking_date": booking_date,
        "error": error,
    }

    return render(
        request,
        "rooms/available.html",
        context,
    )
```

В этой функции: `filter()` выбирает бронирования, соответствующие обоим условиям – дата совпадает с выбранной и бронирование не отменено; `values_list("room_id", flat=True)` возвращает последовательность идентифи-

каторов помещений, а не полные объекты, – поскольку `room` является внешним ключом, его идентификатор доступен через поле `room_id`; `exclude()` исключает помещения с этими идентификаторами, а `order_by("name")` сортирует оставшиеся. Проверка даты в представлении нужна даже при использовании HTML-поля `type="date"`: запрос может быть сформирован вручную.

Полный вариант шаблона `rooms/available.html`:

```
<h1>Свободные помещения</h1>

<form method="get">
    <label for="date">Дата:</label>
    <input
        type="date"
        id="date"
        name="date"
        value="{{ booking_date }}"
    >
    <button type="submit">Найти</button>
</form>

{% if error %}
<p>{{ error }}</p>
{% endif %}

{% if booking_date and not error %}
<h2>Дата: {{ booking_date }}</h2>

{% for room in rooms %}
<article class="room-card">
    <h3>{{ room.name }}</h3>
    <p>Вместимость: {{ room.capacity }} человек</p>
</article>
{% empty %}
<p>На выбранную дату свободных помещений нет.</p>
{% endfor %}
{% endif %}
```

Условие `{% if booking_date and not error %}` позволяет выводить результаты только при наличии даты и отсутствии ошибки её проверки. Если дата не указана, отображается только форма поиска; если дата некорректна, выводится сообщение об ошибке вместо сообщения об отсутствии свободных помещений. Цикл {% for %} с блоком `{% empty %}` выводит сообщение, когда свободных помещений нет.

Запустите сервер и откройте `/rooms/available/`. Проверьте последовательно:

- дата не указана – отображается форма поиска, результаты не выводятся;

- дата указана – выводятся помещения, не занятые активными бронированиями на эту дату;
- помещение занято – помещение с активным бронированием на выбранную дату отсутствует в результатах;
- бронирование отменено – помещение может появиться в результатах, если на эту дату нет других активных бронирований;
- свободных помещений нет – выводится соответствующее сообщение;
- некорректная дата (например, `?date=не-дата`) – выводится сообщение «Укажите корректную дату».

Примечание. Для просмотра и редактирования записей базы данных рекомендуется установить расширение **SQLite Explorer** из Marketplace VS Code

Шаг 4. Завершение этапа. После замены всех функций удалите из `rooms/views.py` неиспользуемые элементы прежней реализации (импорты `sqlite3`, константу `DB_PATH`, функции `load_rooms` и `find_room_by_id`), если они больше нигде в файле не используются. Итоговый набор импортов:

```
from datetime import datetime

from django.shortcuts import get_object_or_404, render

from rooms.models import Room
from bookings.models import Booking
```

Аналогично переведите представления приложения `bookings`: перевод списка, карточки бронирования и статистики описан в подразделе 5.12, а фильтрация, сортировка и постраничный вывод списка – в подразделах 5.13–5.15.

Необходимо адаптировать все представления, использующие прежние SQL-запросы:

- список помещений;
- детальную страницу помещения;
- список бронирований;
- детальную страницу бронирования;
- поиск свободных помещений;
- страницу статистики.

При переработке не следует без необходимости изменять имена URL, шаблоны, навигацию и стили. Цель ПР9 – изменить способ работы с данными, сохранив существующую функциональность.

Примечание. Имена функций представлений, маршрутов, шаблонов и переменных контекста в индивидуальном проекте могут отличаться от примера Roomly. Перевод на ORM выполняется для представлений собственного проекта: сохраняются фактические имена, маршруты и шаблоны, а заменяется только способ получения данных – с прямых SQL-запросов и файлов на обращение к моделям ORM.

5.12. Адаптация представлений бронирований

Представления приложения `bookings` переводятся по той же схеме, что и представления помещений: анализ прежней реализации, замена источника данных на модели ORM и проверка каждого шага. В приложении три представления: список бронирований, карточка бронирования и статистика.

Шаг 1. Проверка маршрутов. Откройте файл `bookings/urls.py` и убедитесь, что маршруты соответствуют представлениям:

```
from django.urls import path

from . import views

app_name = "bookings"

urlpatterns = [
    path("", views.booking_list, name="bookings"),
    path("statistics/",
         views.statistics,
         name="statistics"),
    path("<int:booking_id>/",
         views.booking_detail,
         name="detail"),
]
```

Маршрут `statistics/` объявлен раньше динамического маршрута `<int:booking_id>/`. Конвертер `int` не пропустит слово `statistics`, поэтому в текущем виде конфликта нет, однако правило «статические маршруты раньше динамических» страхует от неоднозначности при последующем изменении конфигурации.

Шаг 2. Перевод списка бронирований. Прежняя функция выполняет SQL-запрос к исходной базе и объединяет три таблицы:

- `bookings` – сведения о бронированиях;
- `rooms` – название и вместимость помещения;
- `users` – имя пользователя.

В выборку попадают только активные бронирования (is_cancelled = 0), результат сортируется по дате бронирования. Эти же условия и структура данных, передаваемых шаблону, должны сохраниться при переходе на ORM.

Замените функцию списка следующим кодом:

```
from django.db.models import F

from bookings.models import Booking
```

```
def booking_list(request):
    # Получаем только активные бронирования.
    # Через связанные модели выбираем название
    # и вместимость помещения, а также имя
    # пользователя.
    bookings_data = (
        Booking.objects
        .filter(is_cancelled=False)
        .annotate(
            room_name=F("room__name"),
            capacity=F("room__capacity"),
            user_name=F("user__name"),
        )
        .values(
            "id",
            "booking_date",
            "room_name",
            "capacity",
            "user_name",
        )
        .order_by("booking_date")
    )
    # Передаём данные в существующий шаблон.
    return render(
        request,
        "bookings/booking_list.html",
        {"bookings": bookings_data},
    )
```

Как работает запрос:

- `filter(is_cancelled=False)` оставляет только активные бронирования;
- `F("room__name")` обращается к полю `name` связанного помещения; `F("room__capacity")` и `F("user__name")` получают вместимость и имя пользователя;
- `annotate()` создаёт дополнительные поля с привычными для шаблона именами room_name, capacity и user_name;
- `values()` возвращает записи в виде словарей с указанными ключами;
- `order_by("booking_date")` сортирует результат по дате.

Такой вариант сохраняет ключи, которые ранее возвращал SQL-запрос, поэтому шаблон `booking_list.html` менять не требуется: выражения вида `{{ booking.room_name }}` и `{{ booking.user_name }}` продолжают работать. Запу-

стите сервер и откройте страницу списка бронирований: отображаются активные бронирования с помещением, вместимостью, пользователем и датой; отменённые записи отсутствуют; записи отсортированы по дате.

Шаг 3. Перевод карточки бронирования. Прежняя реализация `booking_detail()` загружает данные из JSON-файлов и ищет бронирование функцией `find_booking_by_id()`. После перехода на ORM бронирование получается из модели `Booking`, а связанные помещение и пользователь – через внешние ключи:

```
from django.shortcuts import get_object_or_404, render


def booking_detail(request, booking_id):
    # Получаем бронирование по первичному ключу.
    # Одновременно загружаем связанные объекты
    # помещения и пользователя одним запросом.
    booking = get_object_or_404(
        Booking.objects.select_related("room", "user"),
        pk=booking_id,
    )
    # Передаём бронирование в шаблон.
    context = {
        "booking": booking,
    }
    return render(
        request,
        "bookings/booking_detail.html",
        context,
    )
```

В этой функции: `pk=booking_id` – поиск по первичному ключу; `select_related("room", "user")` – получение связанных объектов одним SQL-запросом с объединением таблиц (подраздел 5.10); `get_object_or_404()` – возврат ответа 404, если бронирование не найдено. Вручную загружать JSON-файлы и вызывать `find_booking_by_id()` больше не требуется.

Карточка бронирования отображает связанные данные: название и вместимость помещения, имя пользователя, дату и статус. Полный вариант шаблона `bookings/booking_detail.html`:

```
{% extends "homepage/base.html" %}

{% block title %}
  {% if booking %}
    Бронирование №{{ booking.pk }} – Roomly
  {% else %}
```

```
    Бронирование не найдено – Roomly
  {% endif %}
{% endblock %}

{% block content %}
{% if booking %}
<div class="card shadow-sm">
  <div class="card-header d-flex
              justify-content-between">
    <span class="text-muted">
      Информация о бронировании
    </span>
    <span class="fw-semibold">№{{ booking.pk }}</span>
  </div>
  <div class="card-body">
    <h1 class="card-title h3 mb-4">
      Бронирование №{{ booking.pk }}
    </h1>
    <p class="card-text">
      <strong>Помещение:</strong>
      {{ booking.room.name }}
    </p>
    <p class="card-text">
      <strong>Вместимость:</strong>
      {{ booking.room.capacity }} человек
    </p>
    <p class="card-text">
      <strong>Пользователь:</strong>
      {{ booking.user.name }}
    </p>
    <p class="card-text">
      <strong>Дата:</strong>
      {{ booking.booking_date|date:"d.m.Y" }}
    </p>
    <p class="card-text mb-4">
      <strong>Статус:</strong>
      {% include "bookings/includes/booking_status.html" %}
    </p>
    <a href="{% url 'bookings:bookings' %}"
       class="btn btn-outline-secondary">
      ← К списку бронирований
    </a>
  </div>
</div>
{% else %}
<div class="card border-danger shadow-sm">
```

```
  <div class="card-body">
    <h1 class="card-title h4 text-danger">
      Бронирование не найдено
    </h1>
    <p class="card-text">
      Бронирование с указанным идентификатором
      отсутствует.
    </p>
    <a href="{% url 'bookings:bookings' %}"
       class="btn btn-outline-secondary">
      ← К списку бронирований
    </a>
  </div>
</div>
{% endif %}
{% endblock %}
```

Обращения `{{ booking.room.name }}` и `{{ booking.user.name }}` работают через внешние ключи: ORM получает связанные объекты без ручного JOIN. Фильтр `d.m.Y` отображает дату в привычном формате. Статус выведен отдельным фрагментом `bookings/includes/booking_status.html`:

```
{% if booking.is_cancelled %}
<span class="badge bg-danger">
  Бронирование отменено
</span>
{% else %}
<span class="badge bg-success">
  Бронирование активно
</span>
{% endif %}
```

Отдельный фрагмент удобен тем, что один и тот же индикатор статуса используется и в списке, и в карточке: изменение оформления выполняется в одном месте. Если в шаблоне уже есть собственное оформление карточки, сохраните его, заменив только обращения к данным, которые требуют корректировки.

Проверьте карточку: страница существующего бронирования (например, `/bookings/1/`) отображает помещение, вместимость, пользователя, дату и статус; запрос `/bookings/999999/` возвращает ответ `404`.

Шаг 4. Перевод статистики. Это наиболее содержательная часть перехода: прежний SQL-запрос не просто выбирает записи, а подсчитывает количество активных бронирований для каждого помещения. В исходном запросе использовались:

- `LEFT JOIN` – чтобы помещения без бронирований тоже присутствовали в статистике;

- `COUNT(bookings.id)` – подсчёт бронирований;
- условие `bookings.is_cancelled = 0` – учёт только активных бронирований;
- `GROUP BY` – группировка по помещению;
- `ORDER BY ... DESC` – сортировка по убыванию количества бронирований.

При переходе на ORM важно сохранить все эти свойства. Если просто применить `filter(is_cancelled=False)` к модели `Booking`, помещения без бронирований исчезнут из результата: фильтр сузит исходный набор записей. Поэтому статистику следует строить от модели `Room` и подсчитывать связанные бронирования.

Добавьте импорты в начало `bookings/views.py` (объедините их с существующими, не дублируя):

```
from django.db.models import Count, F, Q

from rooms.models import Room
def statistics(request):
    # Получаем все помещения, включая те, у которых
    # нет активных бронирований.
    statistics_data = (
        Room.objects
        .annotate(
            bookings_count=Count(
                "bookings",
                filter=Q(bookings__is_cancelled=False),
            ),
            room_name=F("name"),
        )
        .values(
            "room_name",
            "bookings_count",
        )
        .order_by("-bookings_count")
    )
    # Передаём статистику в шаблон.
    return render(
        request,
        "bookings/statistics.html",
        {"statistics": statistics_data},
    )
```

Разберите операции ORM:

- `Room.objects` начинает запрос с таблицы помещений, поэтому помещения без бронирований также попадут в результат – аналог `LEFT JOIN`;

- `Count("bookings")` подсчитывает связанные бронирования; обращение `bookings` работает благодаря `related_name="bookings"` в поле `room` модели `Booking`;
- `Q(bookings__is_cancelled=False)` ограничивает подсчёт активными бронированиями; условие применяется внутри агрегатной функции, а не к исходному набору помещений;
- `F("name")` передаёт название помещения под прежним именем `room_name`, чтобы существующий шаблон не пришлось менять;
- `order_by("-bookings_count")` сортирует статистику от большего количества бронирований к меньшему.

Для помещения без активных бронирований значение `bookings_count` будет равно `0`. Общее количество активных бронирований, если оно требуется на странице, получают методом `count()`:

```
active_count = Booking.objects.filter(
    is_cancelled=False,
).count()
```

Проверьте шаблон статистики `bookings/statistics.html`. Если он использует поля `room_name` и `bookings_count`, менять его не нужно. Например, вывод может выглядеть так:

```
<h1>Статистика бронирований</h1>

{% for item in statistics %}
<article>
  <h2>{{ item.room_name }}</h2>
  <p>Активных бронирований:
     {{ item.bookings_count }}</p>
</article>
{% empty %}
<p>Помещения отсутствуют.</p>
{% endfor %}
```

Запустите сервер и откройте `/bookings/statistics/`. Проверьте:

- отображаются все помещения, включая не имеющие бронирований;
- количество соответствует числу активных бронирований;
- отменённые бронирования не учитываются;
- помещения без активных бронирований имеют значение 0;
- статистика отсортирована по убыванию количества бронирований.

Примечание. Перед использованием этих примеров необходимо сверить их с фактическим содержанием страницы статистики в проекте. Требуется сохранить ранее реализованные показатели и правила их расчёта, а не просто заменить прежний запрос любым примером агрегации.

Шаг 5. Очистка импортов и итоговая проверка. После перевода всех трёх функций удалите из `bookings/views.py` элементы прежней реализации, если они больше нигде в файле не используются:

```
# Больше не используются: удаляем
import sqlite3
from storage import load_bookings, load_rooms, load_users

DB_PATH = "data/roomly.sqlite"
```

Не удаляйте элементы, если другие функции в том же файле по-прежнему от них зависят. В результате в файле должны остаться только используемые импорты Django и моделей, например:

```
from django.db.models import Count, F, Q
from django.shortcuts import get_object_or_404, render

from bookings.models import Booking
from rooms.models import Room
```

Выполните `python manage.py check` и последовательно проверьте страницы:

- `/bookings/` – список активных бронирований с фильтрами (подразделы 5.13–5.15);
- `/bookings/1/` – карточка существующего бронирования;
- `/bookings/999999/` – обработка отсутствующей записи;
- `/bookings/statistics/` – статистика по всем помещениям.

В завершение убедитесь, что в `bookings/views.py` больше нет прямых SQL-запросов к `data/roomly.sqlite` и загрузки данных из JSON-файлов. Результат шага: представления приложения bookings получают данные средствами Django ORM, маршруты, имена данных в контексте и логика отображения сохранены, а связанные объекты загружаются средствами ORM.

5.13. Фильтрация списка бронирований по параметрам запроса

Список бронирований становится полезнее, когда пользователь может отобрать интересующие записи: выбрать дату, помещение или пользователя. Параметры фильтрации передаются через строку запроса (`request.GET`) и применяются к выборке последовательно.

Важная особенность Django ORM: фильтровать записи можно по полям связанных моделей через внешние ключи, например `room_id` и `user_id`. Ручное написание SQL и объединение таблиц (JOIN) при этом не требуются.

В файле `bookings/views.py` добавьте чтение параметров запроса и цепочку фильтров к списку активных бронирований:

```
from datetime import datetime
```

```
from django.db.models import F
from django.shortcuts import render
from bookings.models import Booking
from rooms.models import Room
from users.models import User

def booking_list(request):
    booking_date = request.GET.get("date", "").strip()
    room_id = request.GET.get("room", "").strip()
    user_id = request.GET.get("user", "").strip()
    error = None

    # Выбираем только активные бронирования.
    bookings_data = Booking.objects.filter(is_cancelled=False)

    # Фильтрация по дате.
    if booking_date:
        try:
            selected_date = datetime.strptime(
                booking_date, "%Y-%m-%d"
            ).date()
        except ValueError:
            error = "Укажите корректную дату."
        else:
            bookings_data = bookings_data.filter(
                booking_date=selected_date
            )

    # Фильтрация по помещению и по пользователю.
    if room_id.isdigit():
        bookings_data = bookings_data.filter(room_id=int(room_id))
    if user_id.isdigit():
        bookings_data = bookings_data.filter(user_id=int(user_id))
```

Чтобы передать в шаблон названия помещения и пользователя вместо идентификаторов, применяются `annotate()` и `values()`:

```
    bookings_data = (
        bookings_data
        .annotate(
            room_name=F("room__name"),
            capacity=F("room__capacity"),
            user_name=F("user__name"),
        )
        .values(
            "id", "booking_date",
```

```
            "room_name", "capacity", "user_name",
        )
        .order_by("booking_date", "id")
    )
```

В контекст передаются данные для формы фильтрации – списки помещений и пользователей и выбранные значения:

```
    context = {
        "bookings": bookings_data,
        "booking_date": booking_date,
        "selected_room": room_id,
        "selected_user": user_id,
        "rooms": Room.objects.order_by("name"),
        "users": User.objects.order_by("name"),
        "error": error,
    }
    return render(request, "bookings/booking_list.html", context)
```

В шаблоне `bookings/templates/bookings/booking_list.html` добавьте форму с полями фильтрации. Списки вариантов формируются циклом по `rooms` и `users`:

```
<form method="get" class="mb-4">
    <div class="row g-3 align-items-end">
        <div class="col-md-3">
            <label          for="date"          class="form-label">Дата
бронирования</label>
            <input
                type="date"
                id="date"
                name="date"
                class="form-control"
                value="{{ booking_date }}"
            >
        </div>
        <div class="col-md-3">
            <label for="room" class="form-label">Помещение</label>
            <select id="room" name="room" class="form-select">
                <option value="">Все помещения</option>
                {% for room in rooms %}
                <option
                    value="{{ room.pk }}"
                    {% if selected_room == room.pk|stringformat:"s" %}
                    selected
                    {% endif %}
                >
```

```
                    {{ room.name }}
                </option>
                {% endfor %}
            </select>
        </div>
        <div class="col-md-3">
            <label for="user" class="form-label">Пользователь</label>
            <select id="user" name="user" class="form-select">
                <option value="">Все пользователи</option>
                {% for user in users %}
                <option
                    value="{{ user.pk }}"
                    {% if selected_user == user.pk|stringformat:"s" %}
                    selected
                    {% endif %}
                >
                    {{ user.name }}
                </option>
                {% endfor %}
            </select>
        </div>
        <div class="col-md-3">
            <button        type="submit"        class="btn        btn-
primary">Найти</button>
            <a href="{% url 'bookings:bookings' %}"
               class="btn btn-outline-secondary">Сбросить</a>
        </div>
    </div>
</form>

{% if error %}
<p class="text-danger">{{ error }}</p>
{% endif %}
```

Атрибут `selected` отмечает выбранный вариант: сравнение `selected_room == room.pk|stringformat:"s"` приводит идентификатор к строке и сохраняет выбор пользователя после отправки формы; поле даты сохраняет выбор через `value="{{ booking_date }}"`. Цикл вывода списка бронирований ниже формы не меняется.

Метод `values()` возвращает словари, однако обращение к полям в шаблоне по-прежнему записывается через точку: {{ booking.room_name }}, `{{ booking.user_name }}`. Цикл вывода списка бронирований при этом не меняется.

Запустите сервер и откройте страницу `/bookings/`. Проверьте последовательно:

1. Без фильтров отображаются все активные бронирования.
2. При выборе помещения остаются только его активные бронирования.
3. При выборе пользователя остаются его активные бронирования.
4. При одновременном выборе даты, помещения и пользователя отображаются записи, удовлетворяющие всем выбранным условиям.
5. Кнопка «Сбросить» возвращает к списку всех активных бронирований.

Что важно понимать в этом подразделе:

- `filter(room_id=…)` и `filter(user_id=…)` – фильтрация по внешним ключам без ручного SQL и JOIN;
- несколько вызовов `filter()` применяются как условие «И» и последовательно сужают выборку;
- GET-параметры позволяют фильтровать записи без изменения маршрутов и без отдельного представления для каждого фильтра;
- если сочетание фильтров не находит записей, это не обязательно ошибка: проверьте наличие данных, соответствующих всем условиям.

5.14. Сортировка списка бронирований

По умолчанию список бронирований упорядочен по дате. Добавим выбор порядка отображения: по дате, по названию помещения или по пользователю. Значение параметра `sort` читается из запроса, а поле сортировки выбирается из заранее заданного соответствия:

```
    sort = request.GET.get("sort", "date")

    # Добавляем сведения о связанных объектах.
    bookings_data = bookings_data.annotate(
        room_name=F("room__name"),
        capacity=F("room__capacity"),
        user_name=F("user__name"),
    )

    # Выбираем поле сортировки.
    sort_fields = {
        "date": "booking_date",
        "room": "room__name",
        "user": "user__name",
    }

    sort_field = sort_fields.get(sort, "booking_date")

    # Сортируем по выбранному полю,
    # а при совпадении значений — по идентификатору.
```

```
    bookings_data = bookings_data.order_by(sort_field, "id")

    # Формируем набор полей для шаблона.
    bookings_data = bookings_data.values(
        "id",
        "booking_date",
        "room_name",
        "capacity",
        "user_name",
    )

    context = {
        "bookings": bookings_data,
        "booking_date": booking_date,
        "selected_room": room_id,
        "selected_user": user_id,
        "rooms": Room.objects.order_by("name"),
        "users": User.objects.order_by("name"),
        "error": error,
        "sort": sort,
    }

    return render(
        request,
        "bookings/booking_list.html",
        context,
    )
```

Произвольное значение из URL не подставляется в `order_by()` напрямую: словарь `sort_fields` сопоставляет разрешённые значения параметра с полями модели, а при неизвестном значении применяется сортировка по дате. Дополнительное поле `id` обеспечивает стабильный порядок при совпадении значений основного поля.

В существующую форму добавляется выбор сортировки:

```
<div class="col-md-3">
    <label for="sort" class="form-label">Сортировать по</label>
    <select id="sort" name="sort" class="form-select">
        <option value="date"
            {% if sort == "date" %}selected{% endif %}>
            Дате
        </option>
        <option value="room"
            {% if sort == "room" %}selected{% endif %}>
            Помещению
        </option>
        <option value="user"
```

```
            {% if sort == "user" %}selected{% endif %}>
            Пользователю
        </option>
    </select>
</div>
```

В контекст представления добавляется значение `"sort": sort`, чтобы шаблон отмечал выбранный вариант. Сортировка работает совместно с фильтрами подраздела 5.13. Проверьте последовательно:

1. Сортировку по дате.
2. Сортировку по помещению: названия должны идти по алфавиту.
3. Сортировку по пользователю.
4. Одновременное применение фильтра по помещению и сортировки по дате.
5. Кнопку «Сбросить»: список открывается без фильтров, с сортировкой по дате.

5.15. Пагинация списка бронирований

Длинный список удобно разбивать на страницы. В Django для этого служит класс `Paginator` из модуля `django.core.paginator`: он принимает набор данных и количество записей на странице и позволяет получить нужную порцию данных. Подключите класс в `bookings/views.py`:

```
from django.core.paginator import Paginator
```

Пагинация применяется после фильтрации и сортировки, поэтому пользователь получает страницы уже отфильтрованного и упорядоченного списка. Создайте пагинатор и передайте в шаблон объект страницы:

```
    # Создание пагинатора: по 5 записей на страницу.
    paginator = Paginator(bookings_data, 5)

    # Получение текущей страницы из GET-параметра page.
    page_obj = paginator.get_page(request.GET.get("page"))

    context = {
        "bookings": page_obj,
        "page_obj": page_obj,
        "booking_date": booking_date,
        "selected_room": room_id,
        "selected_user": user_id,
        "sort": sort,
        "rooms": Room.objects.order_by("name"),
        "users": User.objects.order_by("name"),
        "error": error,
    }
    return render(request, "bookings/booking_list.html", context)
```

Метод `get_page()` возвращает объект страницы: если параметр `page` отсутствует или содержит некорректное значение, возвращается первая страница, а если номер превышает количество страниц – последняя доступная страница.

Для формирования ссылок навигации используется встроенный тег `{% querystring %}`. Он сохраняет параметры текущего GET-запроса и задаёт новое значение параметра `page`, поэтому выбранная дата, помещение, пользователь и сортировка не теряются при переходе между страницами. Для работы тега объект `request` должен быть доступен в шаблоне – за это отвечает контекстный процессор `django.template.context_processors.request`, подключённый в настройке `TEMPLATES` файла `settings.py` по умолчанию.

После списка бронирований добавьте блок навигации:

```
{% if page_obj.paginator.num_pages > 1 %}
<nav aria-label="Навигация по страницам">
  <ul class="pagination justify-content-center">
    {% if page_obj.has_previous %}
    <li class="page-item">
      <a class="page-link"
         href="{% querystring page=page_obj.previous_page_number %}">
        Назад
      </a>
    </li>
    {% else %}
    <li class="page-item disabled">
      <span class="page-link">Назад</span>
    </li>
    {% endif %}
    {% for page_number in page_obj.paginator.page_range %}
    {% if page_number == page_obj.number %}
    <li class="page-item active" aria-current="page">
      <span class="page-link">{{ page_number }}</span>
    </li>
    {% else %}
    <li class="page-item">
      <a class="page-link"
         href="{% querystring page=page_number %}">
        {{ page_number }}
      </a>
    </li>
    {% endif %}
    {% endfor %}
    {% if page_obj.has_next %}
    <li class="page-item">
      <a class="page-link"
```

```
          href="{% querystring page=page_obj.next_page_number %}">
         Далее
       </a>
     </li>
     {% else %}
     <li class="page-item disabled">
       <span class="page-link">Далее</span>
     </li>
     {% endif %}
   </ul>
</nav>
{% endif %}
```

Блок навигации отображается только тогда, когда количество страниц больше одной. Используются следующие свойства объекта `page_obj`:

- `has_previous` – наличие предыдущей страницы;
- `previous_page_number` – номер предыдущей страницы;
- `number` – номер текущей страницы;
- `paginator.page_range` – последовательность номеров всех страниц;
- `has_next` – наличие следующей страницы;
- `next_page_number` – номер следующей страницы.

Классы `pagination`, `page-item` и `page-link` обеспечивают оформление навигации средствами `Bootstrap`. Проверьте работу пагинации:

1. Список бронирований отображается корректно.
2. Навигация появляется, когда активных бронирований больше пяти.
3. Переход на следующую и предыдущую страницы выполняется корректно.
4. При переходе между страницами сохраняются выбранные фильтры и сортировка.
5. При некорректном значении `page` (например, `/bookings/?page=abc/`) приложение не завершается ошибкой: открывается первая страница.

Примечание. Если в базе мало бронирований, пагинация не проявится визуально: добавьте записи для демонстрации или временно уменьшите размер страницы в вызове Paginator().

5.16. Проверка и сохранение изменений

После перехода к ORM необходимо последовательно проверить проект:

1. Убедитесь, что все модели зарегистрированы.
2. Проверьте, что миграции созданы и применены к ожидаемой базе данных.
3. Проверьте количество записей и связи после переноса данных.
4. Запустите сервер Django.
5. Проверьте открытие главной страницы.
6. Проверьте список и детальную страницу помещений.

7. Проверьте список бронирований и отображение связанных объектов.
8. Проверьте поиск свободных помещений на заданную дату.
9. Проверьте статистику.
10. Проверьте фильтрацию списка бронирований по дате, помещению и пользователю.
11. Проверьте сортировку списка по выбираемому полю.
12. Проверьте постраничный вывод и сохранение параметров фильтрации и сортировки при переходе между страницами.
13. Убедитесь, что отменённые бронирования не блокируют помещение.
14. Проверьте обработку отсутствующих записей и других ошибок.
15. Убедитесь, что представления больше не используют прямые SQL-запросы там, где уже внедрён ORM.
16. Обновите README.md.
17. Зафиксируйте изменения в системе контроля версий.

После завершения работы необходимо выполнить:

```
python manage.py check
python manage.py showmigrations
```

Затем запустить сервер и проверить все реализованные сценарии вручную. При наличии автоматизированных тестов необходимо также выполнить их. Ожидаемый результат команды `check` — сообщение `System check identified no issues (0 silenced)`: оно подтверждает отсутствие выявленных ошибок конфигурации и моделей, но не заменяет проверку страниц в браузере.

Пример сообщения для фиксации изменений:

```
git commit -m "PR9: переход на Django ORM"
```

В README.md необходимо описать новые модели, связи, порядок применения миграций, способ переноса исходных данных и команды запуска проекта. Также следует указать, какая база данных используется приложением после выполнения ПР9. Наброски для файла README.md приведены в подразделе 5.17.

5.17. Актуализация файла README.md

Файл `README.md` сопровождает проект с первых практических работ и должен оставаться актуальным: за время выполнения ПР9 изменились слой данных, структура базы данных и порядок работы с проектом. Требование поддерживать `README.md` в актуальном состоянии закреплено в подразделе 2.5, а обновление файла включено в перечень действий подраздела 5.16. Ниже приведены наброски, которые можно использовать как основу при актуализации файла.

После выполнения ПР9 в README.md целесообразно отразить следующие сведения:

- название проекта и краткое описание предметной области;
- перечень реализованных страниц и пользовательских сценариев;
- технологии: язык Python, фреймворк Django, СУБД SQLite, доступ к данным через Django ORM;
- описание моделей данных: какие сущности они представляют, какие поля и связи имеют;
- структуру каталогов проекта и назначение основных файлов;
- порядок запуска: клонирование репозитория, создание виртуального окружения, установка зависимостей, применение миграций, запуск сервера разработки;
- способ наполнения базы данных исходными данными;
- сведения об авторе проекта.

Возможный каркас файла README.md для проекта Roomly:

```
# Roomly – система бронирования помещений

Краткое описание предметной области и назначения проекта.

## Технологии
- Python 3.x
- Django 5.x
- SQLite (доступ к данным через Django ORM)

## Запуск проекта
1. Клонируйте репозиторий:
   git clone <адрес репозитория>
2. Создайте и активируйте виртуальное окружение:
   python -m venv venv
   venv\Scripts\activate
3. Установите зависимости:
   pip install -r requirements.txt
4. Примените миграции:
   python manage.py migrate
5. Запустите сервер разработки:
   python manage.py runserver
6. Откройте в браузере адрес http://127.0.0.1:8000/

## Модели данных
- Room – помещения; поля: name, capacity
- User – пользователи; поля: name, email
- Booking – бронирования; связи: ForeignKey на Room и User
```

```
## Структура проекта
- rooms/ – модели и представления помещений
- users/ – модель пользователя
- bookings/ – модели и представления бронирований
- templates/ – HTML-шаблоны страниц

## Автор
<имя, группа>
```

Если в проекте не ведётся файл `requirements.txt`, вместо команды установки зависимостей следует указать команды установки используемых библиотек, например `pip install django`.

Локальная база данных, например `db.sqlite3`, обычно не хранится в репозитории: после клонирования таблицы создаются миграциями, а исходные данные наполняются через Django shell либо переносятся средствами dumpdata и loaddata (подраздел 3.6). Если файл базы данных включается в репозиторий, это следует явно указать в README.md.

Примечание. Обновлённый файл README.md включается в итоговый коммит работы (подраздел 5.16). В индивидуальном проекте каркас заполняется сведениями о собственных моделях, приложениях, страницах и командах запуска.

6. РЕЗУЛЬТАТ ВЫПОЛНЕНИЯ ПРАКТИЧЕСКОЙ РАБОТЫ

В результате выполнения ПР9 индивидуальный проект получает слой доступа к данным на основе Django ORM. Используйте приведённые ниже результаты и проверки для самопроверки перед переходом к контролю: каждый пункт должен быть подтверждён демонстрацией или содержимым репозитория.

После выполнения ПР9 в проекте должны быть:

- описаны модели основных сущностей проекта (для `Roomly — Room`, `User`, `Booking`);
- настроены связи между моделями средствами Django ORM;
- созданы и применены миграции;
- данные предыдущей практической работы сохранены или перенесены без потери связей;
- реализованы основные операции с данными через ORM;
- реализованы фильтрация, сортировка и постраничный вывод списка бронирований;
- представления получают данные через ORM и передают их в шаблоны;
- сохранены страницы и пользовательские сценарии предыдущих практических работ;
- поиск свободных помещений учитывает отменённые бронирования;

- статистика соответствует фактическим данным;
- обновлён README.md;
- изменения сохранены в Git.

6.1. Проверка моделей и связей

Необходимо проверить, что модели Django соответствуют сущностям предметной области и связям, предусмотренным проектом.

Проверить:

- наличие моделей `Room`, `User` и `Booking`;
- корректность типов полей и их назначений;
- наличие первичных ключей;
- наличие внешних ключей в модели `Booking`;
- возможность получения помещения и пользователя через объект бронирования;
- возможность получения связанных бронирований через объекты помещения и пользователя;
- корректность метода отмены бронирования, если он реализован в модели.

Пример проверки через Django shell:

```
python manage.py shell

from rooms.models import Room
from users.models import User
from bookings.models import Booking

room = Room.objects.first()
user = User.objects.first()
booking = Booking.objects.select_related(
    "room",
    "user",
).first()

print(room)
print(user)
print(booking)

if booking is not None:
    print(booking.room.name)
    print(booking.user.name)
```

При наличии данных в базе необходимо убедиться, что связанные объекты доступны через соответствующие атрибуты. Если таблицы пусты, сначала следует проверить корректность миграций и переноса данных.

6.2. Проверка миграций

Необходимо убедиться, что:

- файлы миграций созданы;
- миграции сохранены в проекте;
- необходимые миграции применены;
- модели соответствуют текущей структуре базы данных;
- приложение подключается к ожидаемому файлу базы данных;
- существующие данные не были случайно удалены или перезаписаны.

Для проверки использовать команды:

```
python manage.py makemigrations --check
python manage.py showmigrations
python manage.py check
```

Команда `makemigrations --check` позволяет обнаружить изменения моделей, для которых ещё не созданы миграции. Команда `showmigrations` показывает состояние миграций, а `check` проверяет основные настройки проекта.

Если используется отдельная исходная база ПР7 и новая база Django, необходимо дополнительно проверить, что исходный файл сохранён, а данные перенесены в правильную базу.

6.3. Проверка операций ORM

Проверить выполнение основных операций с данными (таблица 6).

Операция	Проверяемый результат
Создание	новая запись появляется в базе данных
Чтение	записи получаются через ORM
Фильтрация	в результат попадают только записи, удовлетворяющие условию
Сортировка	записи расположены в заданном порядке
Изменение	обновлённые значения сохраняются
Удаление	запись удаляется с учётом ограничений связей
Работа со связями	через бронирование доступны помещение и пользователь

Проверки следует выполнять на тестовых данных. Необходимо избегать изменения или удаления исходных данных, если эти операции могут повлиять на демонстрацию проекта.

6.4. Проверка пользовательских сценариев

После изменения представлений необходимо проверить работу страниц, созданных в предыдущих практических работах (таблица 7).

Сценарий	Ожидаемый результат
Открытие главной страницы	страница отображается без ошибок
Просмотр списка помещений	отображаются помещения из базы данных
Просмотр помещения	отображаются сведения о выбранном помещении
Просмотр бронирований	отображаются бронирования и связанные с ними данные
Поиск свободных помещений	выводятся помещения, доступные на заданную дату
Отменённое бронирование	не блокирует помещение на соответствующую дату

Сценарий	Ожидаемый результат
Статистика	отображаются корректные показатели
Несуществующий объект	обрабатывается предусмотренным способом, например возвращается ответ 404
Ошибка ввода	приложение не завершается аварийно, если ошибка относится к проверяемому пользовательскому вводу
Фильтрация списка бронирований	отображаются записи, соответствующие выбранным условиям
Сортировка списка бронирований	записи упорядочены по выбранному полю
Постраничный вывод	список разбит на страницы; фильтры и сортировка сохраняются при переходе

Если в предыдущих практических работах реализованы дополнительные страницы или сценарии, они также должны быть проверены.

6.5. Проверка соответствия кода требованиям

Помимо проверки внешнего поведения приложения, необходимо изучить исходный код. Проверить:

- модели размещены в `models.py` соответствующих Django-приложений;
- связи между моделями описаны средствами Django ORM;
- структура базы данных управляется миграциями;
- получение и изменение данных выполняется через ORM;
- представления не содержат прежних прямых SQL-запросов там, где они заменены ORM;
- шаблоны получают данные через контекст;
- правила предметной области сохранены;
- исходные данные предыдущей практической работы не потеряны;
- README.md соответствует текущему состоянию проекта.

Если в проекте остались прямые SQL-запросы для отдельных задач, необходимо указать, зачем они сохранены. Само по себе использование SQL не является ошибкой во всех возможных случаях, однако в рамках ПР9 основной доступ к описанным моделям должен быть организован через ORM.

7. ФОРМА КОНТРОЛЯ И КРИТЕРИИ ОЦЕНКИ

Защита практической работы проводится в форме демонстрации работоспособности проекта и устного ответа на вопросы, в том числе на контрольные вопросы раздела 8.

Оценивание индивидуального проекта проводится по таблице оценивания (подраздел 7.3), выполнение материалов Практикума подтверждается чек-листом 7.1. Чтобы хорошо подготовиться к защите, необходимо: выполнить пункты чек-листов 7.1 и 7.2 и убедиться, что каждый пункт подтверждается результатом в проекте; пройти по пунктам таблицы оценивания 7.3 и проверить, что каждый показатель можно продемонстрировать и объяснить; подготовить отчёт в соответствии с требованиями подраздела 7.4.

7.1. Чек-лист выполнения материалов Яндекс Практикума

Студент предъявляет преподавателю результат выполнения следующих материалов раздела 05 «База данных и Django ORM». Отмечайте пункты по мере выполнения – чек-лист используется и при самопроверке, и при контроле со стороны преподавателя (таблица 8).

№	Материал Яндекс Практикума	Выполнено
1	Отношения между таблицами	☐
2	Отношения «один к одному»	☐
3	Отношения «многие к одному»	☐
4	Объединение таблиц: JOIN	☐
5	Отношения между таблицами. Многие ко многим	☐
6	Изменение таблиц в БД. Ссылочная целостность	☐
7	Описание таблиц с помощью классов	☐
8	Описание связей между моделями	☐
9	Наследование от абстрактных моделей	☐
10	Миграции	☐
11	Django shell. CRUD-операции в Django ORM	☐
12	Заполнение БД и выгрузка информации из неё	☐

При защите студент должен быть готов подтвердить выполнение материалов курса и продемонстрировать результат соответствующих заданий в проекте.

7.2. Чек-лист выполнения индивидуального проекта

Чек-лист доработки индивидуального сквозного проекта в рамках практической работы № 9 (таблица 9):

№	Требование	Выполнено
1	Проект, разработанный в предыдущих практических работах, сохранён и продолжен	☐
2	Определены сущности предметной области и соответствующие им модели Django	☐
3	Определены поля моделей и выбраны подходящие типы данных	☐
4	Настроены обязательность полей, значения по умолчанию и другие необходимые ограничения	☐
5	Определены первичные ключи моделей	☐
6	Реализованы необходимые связи между моделями с использованием ForeignKey и других подходящих средств ORM	☐
7	Настроены параметры связей, включая on_delete и related_name, если они необходимы	☐
8	Созданы миграции для новых и изменённых моделей	☐
9	Миграции успешно применены к базе данных	☐
10	Проверена возможность получения связанных объектов и обращения к обратным связям	☐
11	Реализовано получение наборов объектов средствами Django ORM	☐
12	Реализовано получение отдельного объекта по идентификатору или другому признаку	☐
13	Реализованы создание, изменение и удаление объектов в соответствии с функциональностью проекта	☐
14	Реализована фильтрация записей по одному условию	☐
15	Реализована фильтрация по нескольким условиям	☐
16	Реализована фильтрация по полям связанных моделей	☐
17	Реализовано исключение записей по заданным условиям с использованием exclude()	☐
18	Реализован поиск по строковому полю модели	☐
19	Реализован поиск по нескольким полям или по дополнительному критерию пред-	☐

№	Требование	Выполнено
	метной области	
20	Реализована сортировка результатов по одному или нескольким полям	☐
21	Реализована выборка данных связанных моделей; при необходимости применены select_related() или prefetch_related()	☐
22	Реализовано получение сокращённого набора полей с использованием values() или другого подходящего средства ORM	☐
23	Реализовано агрегирование данных с использованием aggregate() и агрегатных функций	☐
24	Реализовано аннотирование объектов или группировка результатов с использованием annotate() и подходящих выражений ORM	☐
25	Реализован постраничный вывод результатов с использованием Paginator	☐
26	Реализована навигация между страницами; предусмотрено сохранение параметров поиска, фильтрации и сортировки при переходе между страницами, если эти параметры используются	☐
27	ORM-запросы интегрированы в представления Django, результаты передаются в шаблоны и корректно отображаются в интерфейсе	☐
28	Предусмотрена обработка отсутствующих объектов, пустых результатов и некорректных параметров запроса	☐
29	Проверены модели, связи, миграции, ORM-запросы и основные сценарии работы приложения; выполнена команда python manage.py check	☐
30	Обновлён README.md, изменения сохранены в Git коммитом с сообщением, начинающимся с PR9:	☐
31	Реализовано не менее 15 различных ORM-выражений с соблюдением минимальных требований по видам запросов, установленным в подразделе 2.5 методических указаний	☐

7.3. Таблица оценивания индивидуального проекта

За выполнение работы над индивидуальным проектом (без учёта прохождения курса Яндекс Практикума) студент получает 1,5 балла. Для оценивания используется приведённая ниже таблица из 30 точечных показателей.

Оценивание проводится в форме демонстрации: студент показывает указанные элементы проекта и отвечает на вопросы преподавателя. Опрос проводится выборочно по пунктам таблицы. Демонстрация длится не более 5 минут и завершается на первом неправильном ответе либо его отсутствии; по результатам опроса за студентом закрепляется набранное на этот момент число баллов.

Примечание. В таблице приведены примеры умений и практических задач на материале учебной предметной области Roomly. Демонстрация сформированных умений выполняется на индивидуальном сквозном проекте обучающегося с учётом его предметной области и реализованного функционала.

№	Показатель оценивания	Ожидаемый ответ (результат)
1	Продемонстрируйте описание модели данных, измените тип или параметры поля и объясните результат	В редакторе кода: откройте модели `Room`, `User` и `Booking`. Покажите поля и их параметры. Объясните назначение `blank`, `null` и `default`. При необходимости измените допустимый параметр поля, создайте миграцию и объясните влияние изменения на хранение и проверку данных.

№	Показатель оценивания	Ожидаемый ответ (результат)
2	Продемонстрируйте создание связи между моделями, измените её параметры и объясните результат	В редакторе кода: покажите поля `ForeignKey` модели `Booking`, связывающие бронирование с помещением и пользователем. В Django Shell: получите бронирование и покажите доступ к связанным объектам. Объясните назначение внешних ключей и параметра `on_delete`, а также последствия изменения настройки связи.
3	Продемонстрируйте получение связанных объектов через прямую и обратную связь, измените способ обращения и объясните результат	В Django Shell: покажите обращение к помещению и пользователю через `booking.room` и `booking.user`, а также получение бронирований через обратную связь, если она используется в проекте. Объясните назначение `related_name` и то, как изменится обращение к связанным объектам при изменении этого параметра.
4	Продемонстрируйте создание и применение миграций после изменения модели	В редакторе кода и терминале: покажите модель и соответствующий файл миграции. Выполните `python manage.py makemigrations` и `python manage.py migrate`. Объясните назначение команд, покажите результат применения миграции и объясните, как изменение модели отражается в структуре базы данных.
5	Продемонстрируйте создание объекта модели через ORM, измените значение поля и объясните результат	В Django Shell: создайте тестовый объект `Room`, `User` или `Booking`, измените передаваемое значение до сохранения и сохраните объект. Повторно получите запись из базы данных и покажите результат. Объясните, какие значения были сохранены.
6	Продемонстрируйте изменение существующего объекта через ORM, измените присваиваемое значение и объясните результат	В Django Shell: получите существующий объект, измените одно из его полей и вызовите `save()`. Повторно получите запись и покажите новое значение. Объясните, чем изменение объекта в памяти отличается от сохранения изменений в базе данных.
7	Продемонстрируйте удаление объекта через ORM, измените способ удаления и объясните последствия	В Django Shell: удалите специально созданную тестовую запись методом `delete()`. Покажите, что запись больше не доступна в базе данных. Объясните, как настройки связей и `on_delete` влияют на удаление связанных объектов.
8	Продемонстрируйте получение записей через QuerySet, добавьте условие фильтрации и объясните результат	В редакторе кода или Django Shell: покажите получение записей через `Room.objects.all()` или `Booking.objects.all()`, затем добавьте `filter()`. Измените условие и объясните, какие записи попадут в выборку.
9	Продемонстрируйте фильтрацию по полю связанной модели, измените путь обращения к полю и объясните результат	В редакторе кода или Django Shell: покажите запрос с обращением к связанному полю, например `room__name` или `user__name`. Измените условие и покажите новую выборку. Объясните, как ORM обращается к связанным таблицам и чем фильтрация по связанному полю отличается от фильтрации по внешнему ключу.
10	Продемонстрируйте применение нескольких условий фильтрации, измените одно из условий и объясните результат	В редакторе кода: покажите ORM-запрос для списка бронирований, который учитывает дату, помещение, пользователя или статус. В браузере измените один из фильтров и сравните результаты. Объясните, как каждое условие влияет на состав выборки.

№	Показатель оценивания	Ожидаемый ответ (результат)
11	Продемонстрируйте преобразование пользовательского ввода и обработку некорректных данных	В редакторе кода: покажите получение параметров фильтрации из `request.GET` и их проверку в представлении списка бронирований. В браузере задайте корректные и некорректные значения, например неправильную дату. Покажите предусмотренный результат и объясните, как приложение обрабатывает некорректный ввод.
12	Продемонстрируйте сортировку результатов ORM-запроса, измените поле или направление сортировки и объясните результат	В редакторе кода: покажите применение `order_by()` в запросе списка бронирований. Измените поле сортировки или направление с помощью префикса `-`. В браузере сравните порядок записей и объясните результат изменения запроса.
13	Продемонстрируйте выбор поля сортировки по параметру запроса, измените значение по умолчанию и объясните результат	В редакторе кода: покажите словарь разрешённых полей сортировки и использование параметра `sort`. В браузере измените параметр сортировки и проверьте результат. Объясните, как обрабатывается неизвестное значение и почему нельзя без проверки передавать произвольный пользовательский ввод в `order_by()`.
14	Продемонстрируйте выбор отдельных полей через `values()`, добавьте поле и объясните результат	В редакторе кода: покажите запрос списка бронирований, в котором через `values()` выбираются необходимые поля, включая данные помещения и пользователя. В Django Shell: выполните аналогичный запрос, покажите структуру возвращаемых словарей и объясните, как изменится результат при добавлении или удалении поля.
15	Продемонстрируйте добавление вычисляемого поля через `annotate()` и `F()`, измените выражение и объясните результат	В редакторе кода: покажите применение `annotate()` и `F()`, например `room_name=F("room__name")` или `user_name=F("user__name")`. Покажите, где вычисляемые поля используются далее, например в `values()` и шаблоне. Измените выражение или добавьте поле и объясните, как ORM получает значение из связанной таблицы.
16	Продемонстрируйте получение объекта с обработкой ситуации его отсутствия, измените идентификатор и объясните результат	В редакторе кода: покажите применение `get_object_or_404()` в представлении подробной страницы помещения или бронирования. В браузере откройте существующий объект, затем обратитесь к несуществующему идентификатору. Покажите ответ 404 и объясните, почему отсутствие записи должно обрабатываться без необработанного исключения.
17	Продемонстрируйте подсчёт записей через агрегирующие функции, измените условие агрегации и объясните результат	В редакторе кода: покажите используемые в проекте `count()`, `aggregate()`, `Count()` или условные выражения `Q()`. В браузере откройте страницу статистики и сопоставьте результаты с данными базы. Объясните, какие записи подсчитываются и как изменение условия влияет на результат.
18	Продемонстрируйте формирование статистики по связанным объектам, измените сортировку и объясните результат	В редакторе кода или Django Shell: покажите реализованный запрос статистики по бронированиям или помещениям, использующий группировку или агрегирование, если оно предусмотрено. Измените сортировку и покажите результат. Объясните, какие показатели рассчитываются и как меняется порядок результатов.

№	Показатель оценивания	Ожидаемый ответ (результат)
19	Продемонстрируйте получение связанных объектов через `select_related()`, измените набор связей и объясните результат	В редакторе кода: покажите запрос подробной страницы бронирования с `select_related("room", "user")`. Объясните, какие связанные объекты загружаются вместе с бронированием. В Django Debug Toolbar откройте вкладку SQL и покажите запрос с соединениями таблиц помещения и пользователя. Объясните, как это помогает сократить количество запросов.
20	Продемонстрируйте изменение ORM-запроса для получения только необходимых данных и объясните результат	В редакторе кода: покажите запрос списка бронирований с `values()` и `annotate()`, который получает названия помещения и пользователя. В Django Debug Toolbar покажите соответствующий SQL-запрос. Объясните, какие данные получает приложение и почему для словарей, возвращаемых `values()`, не требуется дополнительно применять `select_related()`.
21	Продемонстрируйте передачу результатов ORM-запроса из представления в шаблон, измените имя переменной и объясните результат	В редакторе кода: покажите получение данных в представлении и передачу через `context`. Затем покажите использование переменной в шаблоне списка помещений или бронирований. Согласованно измените имя переменной в представлении и шаблоне и объясните, что произойдёт, если изменить его только в одном месте.
22	Продемонстрируйте настройку пагинации, измените количество записей на странице и объясните результат	В редакторе кода: покажите использование `Paginator` в представлении списка бронирований. Измените размер страницы и объясните, как изменятся количество страниц и число отображаемых записей. В браузере покажите результат пагинации.
23	Продемонстрируйте реализацию навигации по страницам, измените номер страницы и объясните результат	В редакторе кода: покажите использование `page_obj`, `has_previous` и `has_next` в шаблоне пагинации. В браузере перейдите на следующую и предыдущую страницы. Объясните, при каких условиях отображаются ссылки перехода и как обрабатывается первая или последняя страница.
24	Продемонстрируйте сохранение параметров фильтрации и сортировки при переключении страниц, измените параметр и объясните результат	В редакторе кода: покажите фактически используемый в проекте механизм формирования ссылок пагинации с сохранением GET-параметров, например `{% querystring page=page_number %}`, если он применяется. В браузере задайте фильтр и сортировку, перейдите на другую страницу и объясните, как сохраняются остальные параметры запроса.
25	Продемонстрируйте совместную работу фильтрации, сортировки и пагинации, измените один из параметров и объясните результат	В браузере: задайте фильтр по дате или помещению, выберите сортировку и перейдите на следующую страницу списка бронирований. Измените фильтр или сортировку и покажите обновлённый результат. Объясните, как последовательность обработки запроса влияет на состав и порядок отображаемых записей.
26	Продемонстрируйте изменение запроса для сохранения объектов без связанных записей и объясните результат	В редакторе кода или Django Shell: покажите ORM-запрос, который позволяет включать в результат объекты без связанных записей, если такой сценарий реализован. Объясните, как тип соединения или используемый ORM-подход влияет на результат. Если отдельный сценарий не предусмотрен проектом, продемонстрируйте другой реализованный запрос по связанным моделям.

№	Показатель оценивания	Ожидаемый ответ (результат)
27	Продемонстрируйте использование `exclude()` для исключения записей из выборки, измените условие и объясните результат	В редакторе кода или Django Shell: покажите применение `exclude()` для исключения бронирований по статусу или другому условию. Измените условие и объясните, какие записи исключаются из результата.
28	Продемонстрируйте ограничение количества возвращаемых записей, измените параметры запроса и объясните результат	В редакторе кода или Django Shell: покажите ограничение выборки через срез QuerySet, например `[:10]`, если он используется в проекте. Измените границы среза и объясните, сколько записей будет возвращено. Если ограничение реализовано через пагинацию, покажите соответствующий механизм.
29	Продемонстрируйте получение одной записи через `get()`, измените условие поиска и объясните результат	В Django Shell: покажите получение одного объекта через `get()`. Измените условие поиска и объясните поведение при отсутствии записи и при наличии нескольких записей, удовлетворяющих условию. При необходимости сравните этот способ с `get_object_or_404()` в представлении.
30	Продемонстрируйте совместное использование связанных моделей и ORM-запросов в представлении, измените запрос и объясните результат	В редакторе кода: покажите представление Roomly, которое получает данные через ORM и передаёт их в шаблон. Измените условие фильтрации или способ получения связанных данных, затем проверьте результат в браузере. Объясните, как изменение запроса влияет на отображаемую информацию и почему переход на ORM не должен нарушать существующие пользовательские сценарии.

7.4. Требования к отчету

Объем отчета – 1–2 страницы, не считая титульной информации. Отдельное подробное описание каждого упражнения Яндекс Практикума не требуется. Отчет должен содержать:

1. Общие сведения:

- наименование образовательной организации;
- дисциплину;
- номер практической работы;
- тему практической работы;
- учебную группу;
- ФИО студента;
- вариант.

2. Цель работы. Кратко указать цель освоения моделей Django, связей между ними и операций с базой данных через ORM и их применения в индивидуальном проекте.

3. Выполненные материалы Яндекс Практикума:

Привести перечень выполненных материалов ПР9 с отметкой о выполнении:

Раздел	Выполненные темы
Раздел 05. «База данных и Django ORM»	«Отношения между таблицами»; «Django ORM. Модели»

4. Доработка индивидуального проекта. Указать:

- описанные модели и их поля;
- настроенные связи между моделями;
- созданные и применённые миграции;
- выбранную стратегию перехода от базы ПР7 и способ переноса данных, если он выполнялся;
- реализованные операции ORM;
- адаптированные представления и страницы проекта;
- обновлённый README.md.

5. Результат работы. Кратко указать, что в результате выполнения ПР9:

- описаны модели сущностей проекта;
- настроены связи между моделями;
- созданы и применены миграции;
- данные перенесены без потери связей;
- операции с данными выполняются средствами ORM;
- страницы и пользовательские сценарии проекта работают без изменений.

6. Проверка. Привести результаты:

```
python manage.py check;
python manage.py showmigrations;
pytest -v;
flake8 . --extend-exclude=venv.
```

7. Git. Указать созданные коммиты ПР9.

8. Вывод. Сделать краткий вывод о результатах выполнения практической работы.

К работе прилагаются файлы моделей приложений, файлы миграций, скрипт переноса данных (если он создавался), обновлённый README.md и Git-репозиторий проекта.

8. КОНТРОЛЬНЫЕ ВОПРОСЫ

1. Что такое Django ORM и какую задачу он решает?
2. Как модель Django связана с таблицей реляционной базы данных?
3. Как Django определяет первичный ключ модели, если он явно не объявлен?
4. Чем `CharField`, `TextField`, `IntegerField`, `PositiveIntegerField`, `EmailField` и `BooleanField` отличаются друг от друга?
5. Как в Django реализуется связь «многие к одному»?
6. Для чего используется `ForeignKey`?
7. Что означают параметры `on_delete` и `related_name`?

8. Чем поле модели `room` отличается от атрибута `room_id`?

9. Как получить все записи модели, отфильтровать их и отсортировать?

10. Чем методы `get()` и `filter()` отличаются друг от друга?

11. Для чего применяются `select_related()` и `values_list()`?

12. Чем создание миграций (`makemigrations`) отличается от их применения (`migrate`)?

13. Для чего используется команда `sqlmigrate`?

14. Почему нельзя без проверки применять миграции к базе, созданной до внедрения Django ORM?

15. Какие способы переноса данных в новую структуру базы данных можно использовать?

16. Чем `objects.create()` отличается от создания объекта с последующим вызовом `save()`?

17. Как реализовать поиск свободных помещений средствами ORM?

18. Почему отменённое бронирование не должно блокировать помещение?

19. Как можно получить количество связанных объектов с помощью ORM?

20. Какие изменения необходимо внести в представления и шаблоны при переходе от словарей к объектам моделей?

21. В каких случаях удаление объекта может быть запрещено из-за связанных записей?

22. Почему необходимо проверять не только внешний вид приложения, но и исходный код?

23. Какие сведения необходимо обновить в README.md после перехода к ORM?

24. Какие преимущества и ограничения есть у ORM по сравнению с выполнением SQL-запросов непосредственно через `sqlite3`?

9. ИТОГ

После выполнения ПР9 индивидуальный проект получает слой доступа к данным на основе Django ORM: Реализованы фильтрация, сортировка и постраничный вывод списка бронирований с сохранением параметров при переходе между страницами.

```
Python → Django ORM → SQL → SQLite → объекты моделей
```

Описаны модели основных сущностей предметной области проекта (для Roomly – `Room`, `User` и `Booking`), определены связи «многие к одному» с внешними ключами в модели `Booking`, структура базы данных управляется миграция-

ми, данные перенесены из базы ПР7, а представления получают данные через ORM и передают их в шаблоны. Пользовательские сценарии предыдущих практических работ сохранены и проверены.

Файлы SQL-слоя ПР7 (`sql/schema.sql`, `sql/queries.sql`, `db/database.py`) могут сохраняться в репозитории как предыдущая реализация предметной области: в рамках ПР9 представления переходят на ORM, а прямые SQL-запросы, если они остаются для отдельных задач, должны быть обоснованы.

Таким образом, ПР9 переводит индивидуальный проект на уровень типовой разработки Django-приложений и подготавливает его к дальнейшему развитию (рис. 5):

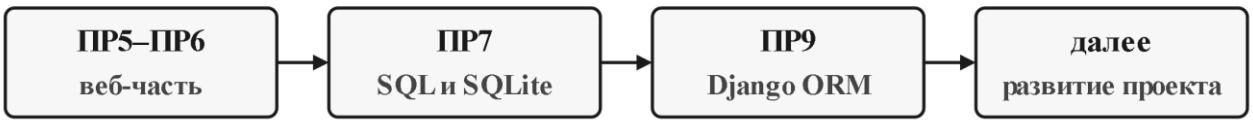


Рисунок 5 – Развитие индивидуального проекта